ŽILINSKÁ UNIVERZITA V ŽILINE

FAKULTA RIADENIA A INFORMATIKY

BAKALÁRSKA PRÁCA

Damián Konečný

Interaktívna aplikácia pre simuláciu a analýzu stratégií v matematickom probléme „100 väzňov“

Vedúci práce: Mgr. Monika Smiešková, PhD.

Registračné číslo: 3759/2025

Žilina, 2026

ŽILINSKÁ UNIVERZITA V ŽILINE

FAKULTA RIADENIA A INFORMATIKY

BAKALÁRSKA PRÁCA

ŠTUDIJNÝ ODBOR: 5ZYI32

Damián Konečný

Interaktívna aplikácia pre simuláciu a analýzu stratégií v matematickom probléme „100 väzňov“

Žilinská univerzita v Žiline
Fakulta riadenia a informatiky
Katedra softvérových technológií
Žilina, 2026

Čestné vyhlásenie

Vyhlasujem, že som pri vypracúvaní predloženej záverečnej práce využil nástroje umelej inteligencie Gemini 1.5 Flash a Gemini 1.5 Pro výhradne na účely gramatickej a štylistickej korektúry textu, generovania námetov na štruktúru práce a technickej konzultácie pri návrhu softvérovej architektúry. Potvrdzujem, že výsledný text práce je mojim autorským dielom, za jeho odbornú náplň nesiem plnú zodpovednosť a žiadna časť práce nebola prevzatá z výstupu umelej inteligencie bez kritického zhodnotenia a riadneho citovania v súlade s etickými princípmi.

Poďakovanie

Týmto spôsobom by som sa chcel poďakovať vedúcej mojej bakalárskej práce, Mgr. Monike Smieškovej, PhD., za prijatie návrhu témy „100 väzňov“, za odborné vedenie, cenné rady a konzultácie, ako aj za jej podporu a ústretový prístup počas celého procesu vypracovania tejto práce.

ABSTRAKT V ŠTÁTNOM JAZYKU

KONEČNÝ, Damián: *Interaktívna aplikácia pre simuláciu analýzu stratégii v matematickom probléme „100 väzňov*. [Bakalárska práca]. – Žilinská univerzita v Žiline. Fakulta riadenia a informatiky; Katedra softvérových technológií. – Vedúci: Mgr. Monika Smiešková, PhD.– Stupeň odbornej kvalifikácie: bakalár. – Žilina: FRI a UNIZA, 2026. Počet strán 90 s.

Bakalárska práca sa zaoberá matematickým problémom „100 väzňov“, ktorý predstavuje fascinujúcu ukážku pravdepodobnosti a teórie cyklov v náhodných permutáciách. Cieľom práce bolo analyzovať optimálnu cyklickú stratégiu a porovnať ju s inými prístupmi prostredníctvom vlastnej softvérovej simulácie. V rámci práce bola v jazyku Java vyvinutá interaktívna aplikácia s grafickým používateľským rozhraním, ktorá umožňuje vizualizáciu algoritmov a štatistické vyhodnocovanie veľkého množstva replikácií. Vďaka integrovaným vizuálnym modulom a vysvetľujúcim prvkom má aplikácia výrazný edukačný charakter, nakoľko pomáha používateľovi intuitívne pochopiť komplexné matematické javy skryté za štruktúrou cyklov. Výsledky simulácií potvrdili teoretický predpoklad, že pri dodržaní cyklickej stratégie sa šanca na prežitie skupiny blíži k hodnote 31,18 %. Práca spája teoretické poznatky z kombinatoriky s praktickou implementáciou algoritmov a spracovaním dát.

Kľúčové slová: problém 100 väzňov, pravdepodobnosť, náhodné permutácie, teória cyklov, simulácia, Java, JavaFx.

ABSTRAKT V CUDZOM JAZYKU

KONEČNÝ, Damián: *Interactive application for simulation and analysis of strategies in the „100 prisoners“ mathematical problem*. [Bachelor's thesis]. – University of Žilina. Faculty of Management Science and Informatics; Department of Software Technologies. – Supervisor: Mgr. Monika Smiešková, PhD. – Degree: Bachelor. – Žilina: FRI and UNIZA, 2026. Number of pages 90 p.

This bachelor's thesis explores the mathematical “100 prisoners” problem, a compelling example of probability and cycle theory in random permutations. The primary objective was to analyse the optimal cycle-following strategy and compare it with alternative approaches using a custom-built software simulation. As part of the project, an interactive application with a graphical user interface was developed in Java, enabling algorithm visualization and statistical evaluation of large-scale replication sets. Due to its integrated visual modules and explanatory elements, the application possesses a significant educational character, allowing users to intuitively grasp the complex mathematical phenomena underlying cycle structures The simulation results confirmed the theoretical prediction that following the cycle strategy increases the survival probability of the group to approximately 31.18 %. The thesis integrates theoretical knowledge of combinatorics with practical algorithm implementation and data processing.

Key words: 100 prisoners problem, probability, random permutations, cycle theory, simulation, Java, JavaFx.

Obsah

Zoznam tabuliek **11**
Zoznam skratiek **12**
Úvod **13**
Motivácia práce 14
Ciele práce 15
1 Súčasný stav riešenej problematiky **16**
1.1 Zahraničie 16
1.2 História a pôvod problému 17
1.3 Definícia a pravidlá problému 18
1.4 Intuitívny vs. Matematický pohľad 18
1.5 Optimálna stratégia teda Stratégia cyklov 19
1.5.1 Matematické vysvetlenie stratégie cyklov 19
1.5.2 Výpočet limitnej pravdepodobnosti 22
1.5.3 Asymptotická analýza pri zvyšovaní počtu väzňov 23
2 Algoritmický prístup a metodika simulácie problému **25**
2.1 Technologický stack a zdôvodnenie výberu jazyka 26
2.2 Algoritmizácia a implementácia optimálnej stratégie 27
2.2.1 Implementácia rozhraní 27
2.2.2 Príprava simulačného prostredia a generovanie krabíc 27
2.2.3 Realizácia cyklickej stratégie 28
2.2.4 Analýza štruktúry cyklov 28
2.3 Náhodná stratégia 29
2.4 Stratégia párnych a neparných čísel 30
2.4.1 Stratégia párnych čísel 30
2.4.2 Stratégia nepárnych čísel (OddStrategy) 31
2.4.3 Stratégia spoločného vylúčenia (ExclusionStrategy) 32
2.4.4 Hybridná stratégia (HybridStrategy) 34
2.5 Architektúra a objektový návrh aplikácie 35
2.5.1 Hierarchia balíkov a komponentov 36
2.5.2 Interakcia medzi komponentmi 38
2.5.3 Správa verzií a archivácia zdrojového kódu 39
2.6 Interaktívne grafické používateľské rozhranie 40
2.6.1 Moduly aplikácie 40
2.6.2 Algoritmická logika pohľadov (View) 40
2.6.3 Permutations View – používateľské rozhranie pre simuláciu 41

2.6.4 Kontrola priebehu a prerušenie simulácie 44
2.6.5 Graph View - grafické zobrazenie výsledkov simulácie 44
2.6.6 Explain View – interaktívne vysvetlenie stratégií 47
2.6.7 Menu View – hlavné navigačné rozhranie aplikácie 52
2.6.8 Vstupné parametre a vizualizácia 55
2.6.9 Interakcia a edukačné prvky systému 55
2.6.10 Prispôsobiteľnosť a rozšíriteľnosť 56
2.7 Metódy verifikácie a testovanie 56
2.7.1 Funkčné testovanie algoritmov UI 56
2.7.2 Porovnávacia verifikácia (Benchmark) 58
2.7.3 Empirické záťažové testovanie 58
2.7.4 Správa verzií ako nástroj verifikácie 59
3 Výsledky práce a diskusia 60
3.1 Realizácia a používateľské rozhranie 60
3.1.1 Hlavné menu a navigácia 60
3.1.2 Modul pre vizuálne vysvetlenie algoritmov 61
3.1.3 Vizualizácia zlyhania pri náhodnej stratégii 62
3.1.4 Vizualizácia paritných stratégií (Párne a nepárne čísla) 64
3.1.5 Vizualizácia stratégie spoločného vylúčenia 66
3.1.6 Vizualizácia hybridnej stratégie 67
3.1.7 Modul pre konfiguráciu a spracovanie simulácií 69
3.1.8 Modul pre grafickú analýzu a štatistiku 71
3.2 Výsledky práce 73
3.2.1 Analýza variability pri opakovaných simuláciách (10 000 replikácií) 73
3.2.2 Analýza stability pri zvýšenom počte replikácií (100 000) 75
3.2.3 Komparatívna analýza presnosti 76
3.2.4 Analýza stability a výpočtovej náročnosti pri 1 000 000 replikáciach 77
3.2.5 Empirické overenie asymptotickej konvergencie (n = 1000) 78
3.2.6 Komparatívne zhodnotenie simulácií (n = 100 vs. n = 1000) 79
3.2.7 Komparatívna analýza s alternatívnymi stratégiami 80
3.3 Diskusia 81
3.3.1 Interpretácia a validácia výsledkov 81
3.3.2 Kognitívny a edukačný prínos vizualizácie 82
3.3.3 Technické zhodnotenie a limity riešenia 82
3.3.4 Prínos pre používateľa a prax 83
Záver 84
Zoznam použitej literatúry 85

Zoznam použitých promptov **86**
Zoznam príloh **87**
Prílohy **88**
Príloha A: Projektová dokumentácia a zdrojové kódy 89
Príloha B: Obsah pamäťového zariadenia 90

Zoznam obrázkov

Obrázok 1 zobrazenie jedného možného cyklu v permutácii 20
Obrázok 2 zobrazenie ako by mohli vyzerať cykly v jednej permutácii 21
Obrázok 3 pravdepodobnosť úspechu viacerých väzňov 23
Obrázok 4 UML diagram tried 36
Obrázok 5 Aplikácia menu 60
Obrázok 6 Grafické vysvetlenie stratégie cyklov 61
Obrázok 7 náhodná stratégia 62
Obrázok 9 stratégia nepárnych čísel 64
Obrázok 8 stratégia párnych čísel 64
Obrázok 10 Stratégia vylúčenia 66
Obrázok 11 hybridná stratégia 67
Obrázok 12 spustenie a výpis simulácie 69
Obrázok 13 výpis simulácie 69
Obrázok 14 zobrazenie behu programu a možnosť zrušenia 70
Obrázok 15 vizualizácia dát pomocou grafov 71
Obrázok 16 vizualizácia dát pomocou grafov 2 71
Obrázok 17 Graf úspešnosti stratégie pre 100 väzňov (N = 10 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti. 74
Obrázok 18 Graf úspešnosti stratégie pre 100 väzňov (N = 100 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti. 75
Obrázok 19 Graf úspešnosti stratégie pre 100 väzňov (N = 1 000 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti. 77
Obrázok 20 Graf úspešnosti stratégie pre 1000 väzňov (N = 1 000 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti. 78

Zoznam tabuliek

Table 1 Porovnanie priemerných výsledkov simulácií a ich odchýlok od teoretických hodnôt pre rôzne konfigurácie experimentu. 80

Zoznam skratiek

Napr. – napríklad

vs. – verzus

Atď. – a tak ďalej

Tzv. – takzvane

Resp. – respektíve

MVC – Model-Pohľad-Riadiaca logika (Model-View-Controller)

Úvod

Práca sa zameriava na analýzu rozporu medzi intuitívnym chápaním problému „100 väzňov“ a jeho presným matematickým riešením. Tento kombinatorický problém, ktorý v roku 2003 sformuloval dánsky informatik Peter Bro Miltersen, predstavuje modelovú situáciu 100 väzňov odsúdených na smrť. V uzavretej miestnosti sa nachádza 100 očíslovaných krabíc, v ktorých sú náhodne poprehadzované čísla väzňov. Každý väzeň musí do miestnosti vstúpiť samostatne a pokúsiť sa nájsť krabicu so svojím číslom, pričom má k dispozícii iba 50 pokusov. Kľúčovou podmienkou je absolútna izolácia – väzni medzi sebou nemôžu počas procesu komunikovať a miestnosť musia po každom pokuse zanechať v pôvodnom stave. Slobodu celá skupina získa len vtedy, ak sa úplne každému jednému väzňovi podarí nájsť svoje číslo; ak zlyhá čo i len jeden z nich, všetci sú popravení. Situácia, ktorá sa na prvý pohľad javí ako prakticky beznádejná, sa pri vhodne zvolenej stratégii ukazuje ako riešiteľná s prekvapivo vysokou pravdepodobnosťou úspechu. Pri čisto náhodnej stratégii je pravdepodobnosť prežitia rovná 0,0000000000000000000000000000079%, zatiaľ čo využitie teórie cyklov v náhodnej permutácii zvyšuje túto hodnotu na približne 31,18%. Tento výrazný rozdiel medzi intuitívnym odhadom a exaktným matematickým výsledkom predstavuje hlavný impulz pre detailnejšie preskúmanie tejto problematiky.

Cieľom bakalárskej práce je podrobná analýza uvedeného problému a demonštrácia princípov prostredníctvom počítačovej simulácie. Pozornosť je venovaná štúdiu cyklov v náhodných permutáciách, ktoré tvoria základ optimálnej stratégie väzňov. Keďže analytické určenie pravdepodobnosti pre veľké systémy môže byť výpočtovo náročné, praktická časť práce sa sústreďuje na implementáciu simulačného softvéru v programovacom jazyku Java.

Okrem teoretického opisu matematického pozadia problému sa práca zameriava aj na overenie zhody teoretických predpokladov s empirickými výsledkami získanými pomocou metódy Monte Carlo. Vytvorený program umožňuje experimentovať s rôznymi parametrami, ako je počet väzňov alebo počet povolených pokusov, a vizualizovať pravdepodobnostné rozdelenie dĺžok cyklov, ktoré priamo ovplyvňujú úspešnosť zvolenej stratégie.

Motivácia práce

V slovenskom akademickom prostredí je „Problém 100 väzňov“ zatiaľ málo známy a nevyskytuje sa bežne vo výučbe matematiky alebo informatiky. Jeho prítomnosť je predovšetkým v rámci vysokoškolských skrípt, zbierok úloh alebo študentských projektov, ktoré sa zaoberajú kombinatorikou a teóriou pravdepodobnosti [7].

Praktické spracovanie problému prostredníctvom počítačových simulácií predstavuje moderný prístup, ktorý umožňuje empiricky skúmať vlastnosti náhodných permutácií a úspešnosť zvolenej stratégie [8]. Predkladaná práca preto nadväzujeme na tento trend a poskytuje nástroj na experimentálne overovanie stratégie sledovania cyklov a vizualizáciu rozdelenia dĺžok cyklov, čím umožňuje lepšie pochopenie matematického fenoménu.

Významným aspektom tejto práce je jej edukačný potenciál, keďže interaktívna aplikácia umožňuje študentom lepšie prepojiť abstraktnú matematickú teóriu s praktickou skúsenosťou. Využitie simulácií vo výučbe pravdepodobnosti a štatistiky pomáha názorne ukázať správanie náhodných javov, pričom študenti môžu sledovať, ako sa experimentálne výsledky postupne približujú k teoretickej hodnote približne 31,18 %.

Grafické znázornenie cyklov zároveň uľahčuje pochopenie rozkladu permutácie, ktorý tak prestáva byť len abstraktným pojmom a stáva sa vizuálne uchopiteľným. Možnosť porovnávať rôzne stratégie podporuje rozvoj algoritmického myslenia a zároveň ukazuje význam matematicky podložených riešení v informatike.

Zároveň tento prístup umožňuje študentom aktívne experimentovať s parametrami problému, ako je počet väzňov alebo nastavenie parametrov simulácie, čím si môžu lepšie osvojiť princípy pravdepodobnosti a pochopiť štruktúru permutácií. Interaktívne simulácie tak slúžia ako efektívny nástroj na prepojenie teoretických poznatkov s praktickou skúsenosťou.

Aplikácia preto neslúži len na overovanie výsledkov, ale aj ako didaktická pomôcka, ktorá podporuje aktívne a experimentálne učenie v oblasti diskrétnej matematiky.

Ciele práce

Hlavným cieľom bakalárskej práce je navrhnúť a implementovať interaktívnu simulačnú aplikáciu v programovacom jazyku Java, ktorá umožní používateľovi skúmať a porovnávať rôzne stratégie riešenia pravdepodobnostného problému 100 väzňov a 100 krabíc. Aplikácia bude slúžiť ako edukačný nástroj na demonštráciu matematických princípov tohto problému a ako prostriedok na empirické overovanie teoretických výsledkov prostredníctvom simulácií.

Na dosiahnutie hlavného cieľa boli stanovené nasledujúce čiastkové ciele:

1. **Analýza matematického problému:** Naštudovať teoretické základy problému 100 väzňov, jeho klasickú formuláciu, optimálnu stratégiu založenú na sledovaní cyklov v permutáciách a jej matematické zdôvodnenie.
2. **Návrh a implementácia simulačného jadra:** Vytvoriť programovú logiku umožňujúcu simuláciu optimálnej stratégie a vybraných alternatívnych stratégií, napríklad náhodného výberu krabíc.
3. **Návrh a implementácia používateľského rozhrania:** Vytvoriť jednoduché interaktívne grafické rozhranie, ktoré obsahuje:
    a. stručné predstavenie a zadanie matematického problému
    b. vysvetlenie teoretického pozadia analyzovaných stratégií
    c. možnosť konfigurácie vstupných parametrov simulácie (typ stratégie, počet väzňov, počet opakovaní simulácie).
4. **Vizualizácia a porovnanie výsledkov:** Zabezpečiť nástroje na vizualizáciu priebehu simulácie a na štatistické vyhodnotenie úspešnosti jednotlivých stratégií, umožňujúce ich vzájomné porovnanie.
5. **Testovanie a analýza získaných dát:** Vykonať experimenty s rôznymi hodnotami vstupných parametrov, analyzovať získané dáta a interpretovať výsledky v súlade s teoretickými predpokladmi.

1 Súčasný stav riešenej problematiky

1.1 Zahraničie

Matematický problém známy ako „Problém 100 väzňov“ patrí medzi známe príklady kontraintuitívnych výsledkov v teórii pravdepodobnosti a kombinatorike. V roku 2003 ho sformuloval dánsky informatik Peter Bro Miltersen na konferencii International Colloquium on Automata, Languages and Programming (ICALP) [1], kde bol prezentovaný v kontexte teoretickej informatiky a analýzy algoritmov. Pôvodne nebol určený ako logická hádanka pre širokú verejnosť, ale ako modelový problém študujúci náhodné permutácie a štruktúru cyklov.

V zahraničnej odbornej literatúre sa problém uvádza ako modelová úloha ilustrujúca vlastnosti náhodných permutácií a ich rozklad na disjunktné cykly. Teoretické pozadie analýzy cyklov je podrobne spracované napríklad v rámci analytickej kombinatoriky, ktorú systematicky rozpracovali Philippe Flajolet a Robert Sedgewick [2]. Ich výsledky poskytujú formálny aparát na štúdium distribúcie dĺžok cyklov v náhodných permutáciách, čo tvorí matematický základ riešenia tohto problému.

Problém sa využíva nielen vo výučbe diskrétnej matematiky a teórie pravdepodobnosti, ale aj v informatike – predovšetkým pri analýze algoritmov a štúdiu výpočtovej zložitosti. Okrem akademického prostredia sa stal predmetom záujmu aj populárno-vedeckých médií a kanálov, ako napríklad YouTube kanál Veritasium [3], ktoré zdôrazňujú kontraintuitívnu povahu tohto problému a demonštrujú hodnotu matematického uvažovania.

V zahraničí vzniklo množstvo softvérových implementácií problému v rôznych programovacích jazykoch (Python, R, Java) [4] [5] [6], ktoré umožňujú vizualizovať cykly a empiricky overovať úspešnosť stratégie. Tieto simulácie sa často využívajú v edukatívnych projektoch, kde študenti môžu experimentovať s rôznymi parametrami, ako je počet väzňov alebo limit otvorených krabíc.

1.2 História a pôvod problému

Teória pravdepodobnosti a kombinatorika patria medzi klasické oblasti matematiky, ktoré sa systematicky rozvíjajú už od 17. storočia. Významný podiel na ich formovaní mali osobnosti ako Blaise Pascal a Pierre-Simon Laplace, ktorí položili základy formálnej analýzy pravdepodobnostných modelov. Napriek bohatej histórii týchto disciplín je konkrétna formulácia „Problému 100 väzňov“ relatívne nová a predstavuje moderný prienik medzi diskrétnou matematikou a teoretickou informatikou.

Ako už bolo spomenuté, dánsky informatik Peter Bro Miltersen tento problém v roku 2003 pôvodne koncipoval ako model v rámci štúdia výpočtovej zložitosti a komunikačných protokolov. Hoci bol problém prvýkrát prezentovaný na konferencii ICALP v technickom kontexte, jeho presah do širšieho povedomia matematickej komunity bol mimoriadne rýchly. Práve vďaka svojej kontraintuitívnej povahe, kde sa zdanlivo beznádejná situácia mení na riešiteľnú, sa tento problém stal atraktívnym a často citovaným príkladom v moderných učebniciach diskrétnej matematiky.

Teoretický aparát potrebný na exaktné riešenie problému čerpá z hlbokých poznatkov analytickej kombinatoriky, ktorej rozvoj v posledných desaťročiach výrazne ovplyvnil najmä Philippe Flajolet. Spolu s Robertom Sedgewickom vytvorili systematický rámec, ktorý umožňuje transformovať kombinatorické problémy na štúdium analytických vlastností generujúcich funkcií. Ich metodológia poskytuje kľúčové nástroje na presné určenie distribúcie dĺžok cyklov v náhodných permutáciách. Práve pochopenie faktu, že náhodná permutácia s vysokou pravdepodobnosťou neobsahuje cyklus dlhší ako 50, je pre analýzu stratégie väzňov nevyhnutné. Tento akademický základ tak prepája čistú matematickú teóriu s praktickou algoritmizáciou, ktorú táto práca ďalej skúma.

1.3 Definícia a pravidlá problému

Pre analýzu možných stratégií je potrebné presne definovať pravidlá, ktoré určujú štruktúru problému:

1. **Označenie väzňov**: Každý zo 100 väzňov má priradené unikátne číslo od 1 do 100.
2. **Príprava experimentu**: V miestnosti sa nachádza 100 krabíc. Dozorca do nich náhodne rozmiestni čísla od 1 do 100 tak, že každé číslo je použité práve raz. Takto vzniká náhodná permutácia množiny 100 prvkov.
3. **Priebeh experimentu**: Väzni vstupujú do miestnosti postupne, každý samostatne. Každý z nich môže otvoriť maximálne 50 krabíc.
4. **Individuálny cieľ**: Každý väzeň musí nájsť krabicu obsahujúcu jeho vlastné číslo.
5. **Kritérium úspechu skupiny**: Experiment je úspešný iba v prípade, že všetci väzni nájdu svoje číslo. Ak zlyhá čo i len jeden z nich, skupina ako celok neuspeje.
6. **Obmedzenie komunikácie**: Väzni sa môžu dohodnúť na spoločnej stratégii pred začatím experimentu. Po vstupe prvého väzňa do miestnosti však nie je dovolená žiadna komunikácia a rozmiestnenie čísel v krabiciach zostáva po každom pokuse nezmenené. Po každom pokuse sú krabice opäť uzavreté a uvedené do pôvodného stavu.

1.4 Intuitívny vs. Matematický pohľad

Z intuitívneho hľadiska sa môže zdať, že každý väzeň má pri náhodnom výbere krabíc pravdepodobnosť $\frac{1}{2}$, že nájde svoje číslo, keďže môže otvoriť polovicu zo všetkých krabíc. Ak by boli pokusy jednotlivých väzňov navzájom nezávislé, celková pravdepodobnosť úspechu skupiny by bola rovná $\left(\frac{1}{2}\right)^{100}$, čo predstavuje hodnotu približne 0,0000000000000000000000000000008 (resp. $8 \cdot 10^{-31}$).

Táto hodnota je v praktickom zmysle nulová a šanca na prežitie skupiny by bola v skutočnosti nižšia než pravdepodobnosť výhry jackpotu v najväčšej svetovej lotérii Powerball (približne $3{,}4 \cdot 10^{-9}$) trikrát po sebe. Tento intuitívny predpoklad však nezohľadňuje štruktúru náhodnej permutácie.

Pri vhodne zvolenej stratégii založenej na sledovaní cyklov v permutácii nie sú pokusy väzňov nezávislé. Úspech alebo neúspech skupiny je determinovaný globálnou vlastnosťou permutácie, konkrétne maximálnou dĺžkou jej cyklov. V dôsledku toho sa celková pravdepodobnosť úspechu výrazne zvyšuje a pre prípad 100 väzňov dosahuje približne 31,18 %.

1.5 Optimálna stratégia teda Stratégia cyklov

Na prekonanie takmer nulovej pravdepodobnosti úspechu pri náhodnom výbere sa väzni musia ešte pred začiatkom pokusu dohodnúť na stratégii založenej na sledovaní cyklov náhodnej permutácie. Obsah krabíc možno interpretovať ako náhodnú permutáciu množiny {1, 2, ..., 100}. Táto stratégia nevyžaduje žiadnu komunikáciu počas samotného priebehu pokusu, a teda neporušuje stanovené pravidlá.

Každý väzeň postupuje podľa nasledujúcich krokov:

1. **Vstupný bod:** Väzeň pristúpi ku krabici, ktorá je označená jeho vlastným číslom.
2. **Otvorenie a kontrola:** Ak krabica obsahuje lístok s jeho číslom, väzeň uspel a opúšťa miestnosť.
3. **Sledovanie ukazovateľa:** Ak krabica obsahuje iné číslo (napr. číslo k), väzeň v ďalšom kroku otvorí krabicu s indexom k.
4. **Opakovanie:** Tento proces opakuje, až kým nenájde svoje číslo alebo kým nevyčerpá limit 50 pokusov.

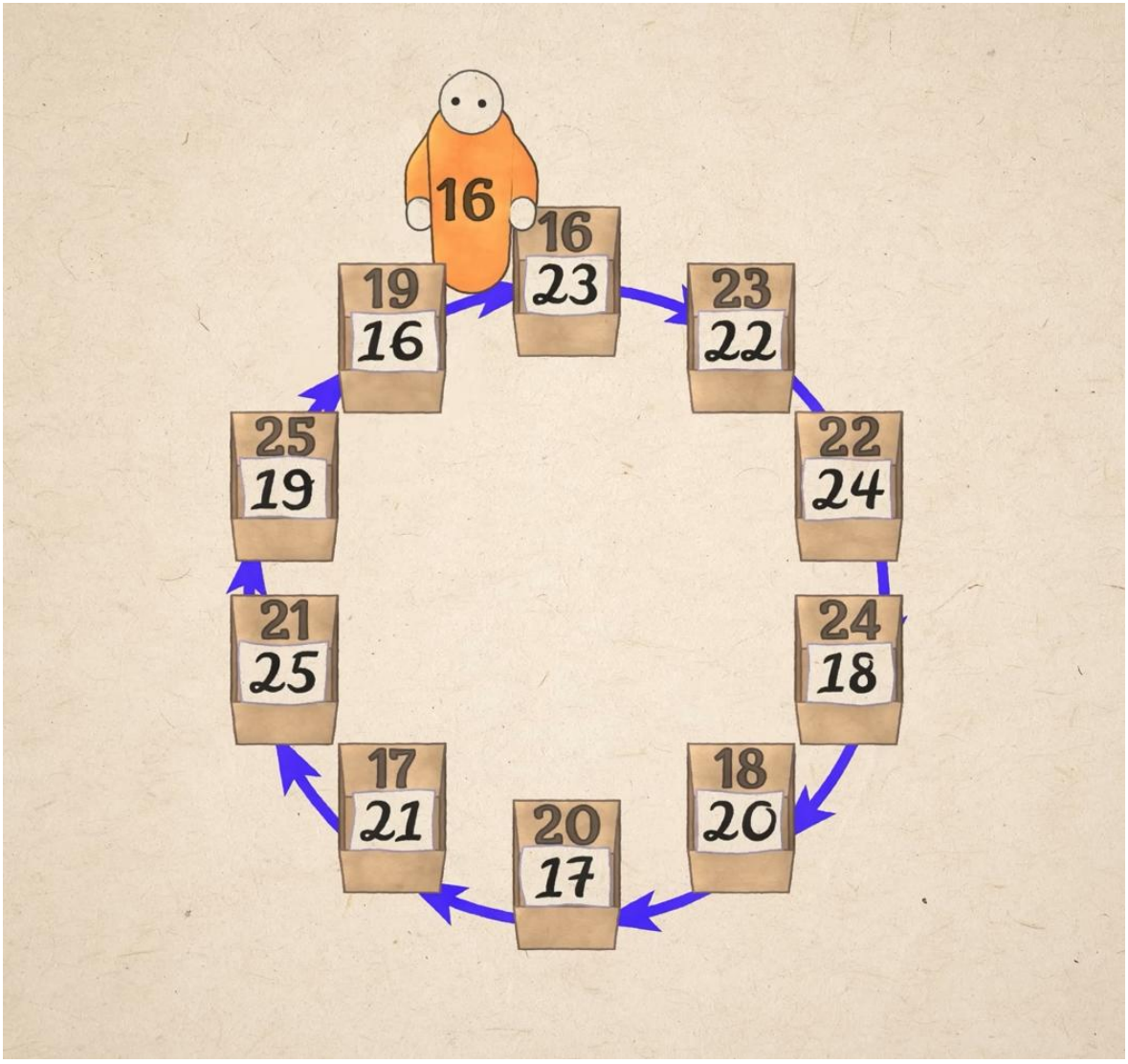
Týmto spôsobom každý väzeň v skutočnosti sleduje cyklus permutácie, do ktorého patrí jeho číslo. Úspech jednotlivca nastane práve vtedy, ak dĺžka príslušného cyklu nepresiahne 50.

1.5.1 Matematické vysvetlenie stratégie cyklov

Sila tejto stratégie spočíva v tom, že väzeň sa neustále pohybuje v rámci jedného konkrétneho cyklu, do ktorého patrí jeho číslo. Keďže permutácia čísel v krabiciach je pre všetkých väzňov rovnaká, úspech skupiny už nie je súčinom 100 nezávislých náhodných udalostí, ale závisí od jedinej globálnej vlastnosti danej permutácie – od dĺžky jej cyklov.

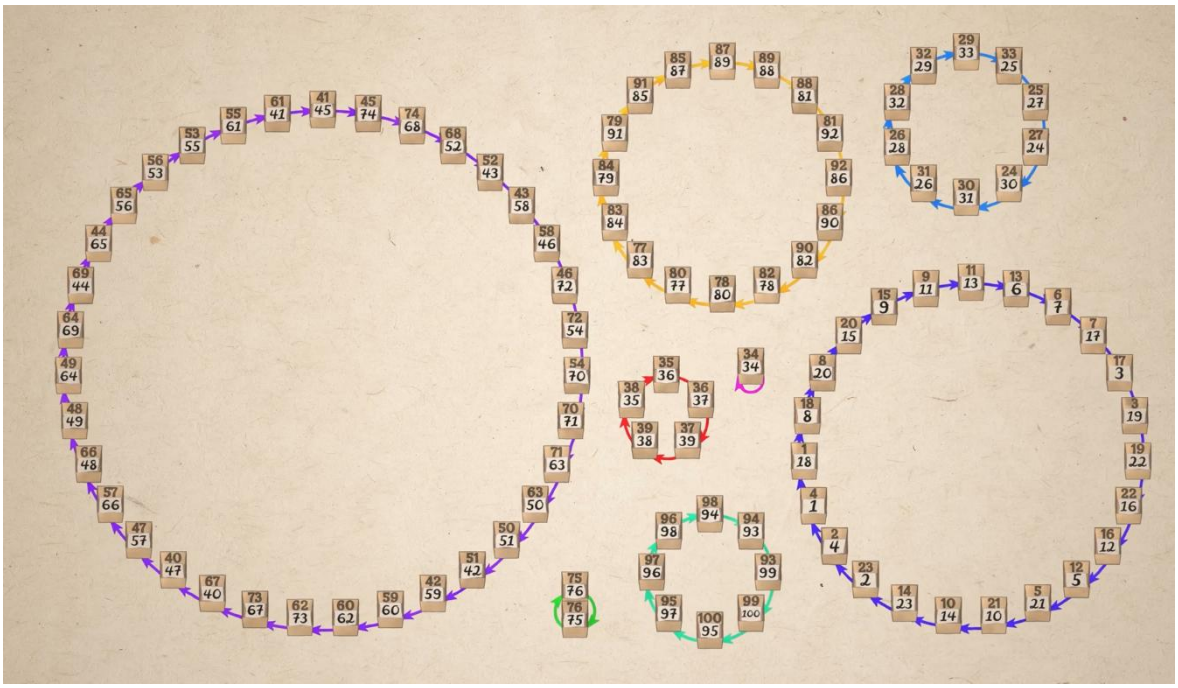
Tento princíp je znázornený na Obrázku 1. V tomto príklade väzeň s číslom 16 začína otvorením krabice so svojim číslom. Vo vnútri nachádza lístok s číslom 23, ktorý ho

odkazuje na krabicu 23. Tento proces pokračuje reťazovo ďalej: 23 – 22 – 24 – 18 – 20 – 17 – 21 – 25 – 19 a napokon 19 – 16, čím sa cyklus uzatvára. Celková dĺžka tohto konkrétneho cyklu je 10, čo znamená, že všetkých 10 väzňov, ktorých čísla patria do tohto cyklu (teda väzni 16, 23, 22, 24, 18, 20, 17, 21, 25 a 19), nájde svoje číslo najneskôr na 10. pokus.



Obrázok 1 zobrazenie jedného možného cyklu v permutácii

Úspech celej skupiny väzňov závisí od toho, ako sa náhodne rozdelí všetkých 100 krabíc do takýchto uzavretých cyklov. Na Obrázku 2 je zobrazený komplexný príklad jednej možnej permutácie 100 prvkov. V tomto prípade sa vytvorilo niekoľko cyklov rôznych dĺžok: jeden veľmi dlhý (fialový), niekoľko stredných (žltý, modrý) a niekoľko veľmi krátkych (červený, zelený, tyrkysový). Kľúčovým pozorovaním je, že skupina prežije len vtedy, ak ani jeden z týchto cyklov nepresiahne dĺžku 50.



Obrázok 2 zobrazenie ako by mohli vyzerať cykly v jednej permutácii

Z uvedeného vyplývajú nasledujúce vlastnosti

- **Závislosť pokusov:** Ak sa čísla väzňa nachádza v cykle dĺžky 30, svoje číslo nájde najneskôr na 30. pokus, Zároveň platí, že všetci ostatní väzni, ktorých čísla patria do toho istého cyklu, takisto úspešne nájdu svoje čísla.
- **Kritická dĺžka cyklu:** Skupina väzňov prežije práve vtedy, ak náhodne vygenerovaná permutácia 100 prvkov neobsahuje žiadny cyklus dlhší ako 50.
- **Pravdepodobnostný zlom:** V permutácii 100 prvkov môže existovať maximálne jeden cyklus dlhší ako 50. Pravdepodobnosť neúspechu je rovná súčtu pravdepodobností že permutácia obsahuje cyklus dĺžky k pre $k = 51, \ldots, 100$.

1.5.2 Výpočet limitnej pravdepodobnosti

Výpočet úspešnosti stratégie sledovania cyklov vychádza z rozkladu náhodnej permutácie na disjunktné cykly. Kľúčovým poznatkom je, že v náhodnej permutácii n prvkov je pravdepodobnosť, že obsahuje cyklus dĺžky L, rovná $\frac{1}{L}$ pre všetky L ∈ {1,2,3, . . . , n}.

Analýza zlyhania cez harmonické čísla

Skupina väzňov zlyhá práve vtedy, ak sa v permutácii vyskytne cyklus s dĺžkou L > $\frac{n}{2}$. Keďže súčet dĺžok všetkých cyklov musí byť presne n, v jednej permutácii môže existovať najviac jeden takýto dlhý cyklus. Pravdepodobnosť neúspechu P(zlyhanie) je teda definovaná ako súčet pravdepodobnosti výskytu všetkých neprípustných dĺžok cyklov:

Pre n = 100

$$P(zlyhanie) = \frac{1}{51} + \frac{1}{52} + \cdots + \frac{1}{100}$$

Tento výraz predstavuje rozdiel medzi čiastočnými súčtami harmonického radu, konkrétne $H_{100} - H_{50}$

Aproximácia a konvergencia

Pre dostatočne veľké n môžeme harmonické číslo H_n aproximovať pomocou prirodzeného logaritmu ako $1 + n \approx ln(n) + \gamma$ (kde γ je Eulerova-Mascheroniho konštanta). Pravdepodobnosť zlyhania pre n väzňov a limit n/2 potom nadobúda tvar:

$$\text{P(zlyhanie)} \approx ln(n) + \gamma - \approx ln\left(\frac{n}{2}\right) + \gamma = ln(n) - ln\left(\frac{n}{2}\right) = ln\frac{\frac{(n)}{n}}{2} = ln(2)$$

Hodnota $ln(2)$ je približne 0,6931. Pravdepodobnosť úspechu skupiny je doplnkovým javom k zlyhaniu, čo vedie k limitnému vzťahu pre $n \to \infty$:

P(úspech) = 1 - $ln(2) \approx 1 - 0{,}6931 = 0{,}3068 \ldots$

Interpretácia pre 100 väzňov

Hoci aproximácia pomocou $ln(2)$ dáva hodnotu približne 30,69 % presný súčet zlomkov pre diskrétny prípad 100 väzňov vedie k hodnote 31,18 %. Tento výsledok je prekvapivý svojou stabilitou, zatiaľ čo pri náhodnom výbere šanca na prežitie s rastúcim n prudko klesá k nule, pri stratégii cyklov neklesne nikdy pod hranicu cca 30,69 % bez ohľadu na to, či ide o stovky alebo milióny väzňov.

1.5.3 Asymptotická analýza pri zvyšovaní počtu väzňov

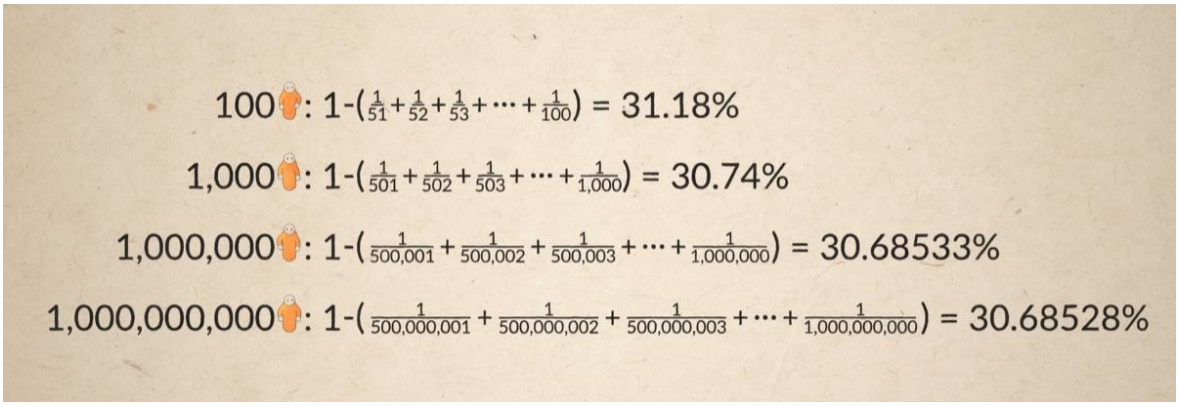
Jednou z najpozoruhodnejších vlastností stratégie sledovania cyklov je jej stabilita nezávislá od veľkosti skúmanej vzorky. Zatiaľ čo pri intuitívnom pohľade by sa mohlo zdať, že so zvyšujúcim sa počtom väzňov pravdepodobnosť na prežitie skupiny výrazne klesá, matematický model dokazuje opak.

Pravdepodobnosť úspechu pri stratégii cyklov je pre ľubovoľné n definované ako:

$$P(úspech) = 1 - \sum_{k=\frac{n}{2}+1}^{n} \frac{1}{k}$$

Ako je znázornené na nasledujúcom obrázku (Obrázok 3), tento vzťah vykazuje silnú tendenciu ku konvergencii k limitnej hodnote 1 - $ln(2)$:

- **Pre n = 100:** Šanca na prežitia je 31,18 %. Tento bod slúži ako základný referenčný rámec pre väčšinu simulácií.
- **Pre n = 1000:** Hodnota mierne klesá na 30,74 %. Ide o pokles o približne 0,44 percentuálneho bodu, čo potvrdzuje, že stratégia ostáva vysoko efektívna aj pri desať násobnom zväčšení skupiny.
- **Pre n = 1 000 000:** Pravdepodobnosť sa ustáli na 30,68533 %. Rozdiel oproti predchádzajúcemu stupňu je už len na úrovni tisícin percenta.
- **Pre n = 1 000 000 000:** Hodnota 30,68528 % demonštruje, že od určitého bodu je vplyv počtu účastníkov na výslednú šancu prakticky zanedbateľný.



Obrázok 3 pravdepodobnosť úspechu viacerých väzňov

Z tejto analýzy a výpočtov vyplývajú nasledujúce kľúčové závery:

- **Ohraničenie zdola:** Funkcia prežitia je zdola ohraničená hodnotou $1 - ln(2) \approx 30{,}685$ %. To znamená, že bez ohľadu na to, akú veľkú skupinu väzňov sa uvažuje, ich šanca na úspech nikdy neklesne pod hranicu približne 30,7 %.
- **Nezávislosť od počtu účastníkov:** Tento poznatok je kľúčový pre praktickú časť práce, pretože predpokladá, že výsledky simulácii pre 100 a 1000 väzňov budú takmer identické. V systéme, kde by sme mali milión alebo dokonca miliardu väzňov, by výsledok simulácie stále vykazoval úspešnosť nad 30 %.
- **Kontrast sa náhodným výberom:** Tento jav ostro kontrastuje so stratégiou náhodného výberu. Pri nej pravdepodobnosť klesá exponenciálne s každým ďalším väzňom ($0{,}5^n$). Kým pri miliarde väzňov je šanca na náhodný úspech prakticky nulová, pri stratégii cyklov je prežitie skupiny stále vysoko pravdepodobné.

Práve táto matematická „istota“ minimálneho 30 % úspechu robí z cyklickej stratégie optimálne riešenie, ktoré popiera bežnú ľudskú intuíciu o narastajúcom riziku vo veľkých skupinách.

2 Algoritmický prístup a metodika simulácie problému

Na dosiahnutie stanovených cieľov bakalárskej práce bolo nevyhnutné zvoliť vhodnú metodiku spracovania danej problematiky a aplikovať vedecké metódy výskumu.

Prvým krokom bolo zhromaždenie a spracovanie informácií z odbornej a vedeckej literatúry. Tento proces poskytol široký prehľad o teórii pravdepodobnosti, kombinatorike a špecificky o „Probléme 100 väzňov“. Literárna rešerš bola realizovaná počas celého procesu spracovania práce, najmä pri štúdiu algoritmov na generovanie náhodných permutácií a ich rozkladu na cykly.

V rámci výskumu boli aplikované viaceré vedecké metódy:

- **Analýza:** Slúžila na identifikáciu hlavných matematických faktorov vedúcich k úspešnej stratégii a na rozčlenenie problému na menšie celky. Bola využitá najmä pri hĺbkovom štúdiu súčasného stavu poznania v oblasti teórie grafov a permutácií.
- **Syntéza:** Umožnila integrovať teoretické poznatky do návrhu simulačnej aplikácie. Matematické vzorce a algoritmické postupy boli zlúčené do uceleného a funkčného programu v jazyku Java.
- **Komparácia**: Bola využitá na porovnanie efektívnosti rôznych stratégií, konkrétne náhodnej stratégie voči stratégii sledovania cyklov. Umožnila analyzovať rozdiely v úspešnosti a identifikovať kľúčové faktory ovplyvňujúce výsledok.
- **Dedukcia:** Aplikovala sa na vyvodenie záverov z dostupných pravdepodobnostných modelov a na formulovanie predpokladov pre simulačnú časť práce.

Okrem uvedených metód boli využité aj deskriptívne a štatistické metódy, najmä pri aplikácii metódy Monte Carlo. Tieto postupy umožnili interpretovať dáta získané z veľkého množstva simulácií a preukázať zhodu empirických výsledkov s teoretickými výpočtami.

2.1 Technologický stack a zdôvodnenie výberu jazyka

Pred samotnou implementáciou algoritmov bolo nevyhnutné zvoliť technológie, ktoré zabezpečia vysoký výpočtový výkon a zároveň umožnia vytvorenie moderného grafického rozhrania. Pre realizáciu simulačného nástroja sme zvolili programovací jazyk Java (verzia JDK 17 a vyššie) v kombinácii s knižnicou JavaFX. Tento výber opierame o nasledujúce kritéria:

- Výkon a správa vlákien (Multithreading): Simulácia miliónov permutácií je výpočtovo náročná operácia. Java poskytuje vyspelé API pre asynchrónne spracovanie dát, čo nám umožnilo oddeliť výpočtové jadro od vykresľovacieho vlákna (JavaFX Application Thread). Výsledkom je plynulé rozhranie, ktoré nezamŕza ani pri extrémnej záťaži.
- Typová bezpečnosť a robustnosť: Keďže pracujeme s komplexnými dátovými štruktúrami (dekompozícia permutácií na cykly), silná statická typová kontrola v Jave minimalizuje riziko logických chýb pri spracovaní indexov a hodnôt v poliach.
- Moderné GUI a deklaratívny prístup: JavaFX umožňuje striktné oddelenie logiky od dizajnu pomocou súborov FXML (XML štruktúra) a štýlovanie cez CSS. To výrazne uľahčilo tvorbu responzívneho analytického dashboardu a interaktívnych histogramov.
- Prenositeľnosť a ekosystém: Architektúra JVM (Java Virtual Machine) zaručuje, že aplikácia je spustiteľná na rožných operačných systémoch bez nutnosti modifikácie kódu, čo je pre edukačný softvér kľúčová vlastnosť.

2.2 Algoritmizácia a implementácia optimálnej stratégie

Na realizáciu optimálnej stratégie založenej na sledovaní cyklov bola implementovaná trieda CycleStrategy. Algoritmus simuluje správanie väzňov presne podľa matematických predpokladov popísaných v teoretickej časti práce a umožňuje empiricky overiť úspešnosť zvoleného prístupu.

2.2.1 Implementácia rozhraní

V systéme je definované rozhranie Strategy, ktoré slúži ako predpis pre všetky implementované algoritmy vyhľadávania. Tento návrh umožňuje jednoduché pridávanie nových stratégií bez potreby modifikácie existujúceho kódu grafického používateľského rozhrania (GUI), čo zodpovedá objektovo-orientovanému princípu otvorenosti a uzavretosti (Open-Closed Principle).

2.2.2 Príprava simulačného prostredia a generovanie krabíc

Prvým krokom algoritmu je inicializácia počiatočného stavu systému. Implementovaná metóda generujKrabice() vytvára zoznam (List<Integer>) obsahujúci unikátne čísla predstavujúce identifikátory väzňov.

Pre zabezpečenie náhodného usporiadania je použitá metóda Collections.shuffle(), ktorá aplikuje Fisher-Yatesov algoritmus. Ten pracuje tak, že postupne prechádza zoznam a pre každý prvok náhodne vyberie iný prvok na výmenu. Týmto spôsobom sa zabezpečí rovnomerné náhodné preusporiadanie všetkých prvkov, pričom každá možná permutácia má rovnakú pravdepodobnosť výskytu.

Rovnomerná náhodná permutácia prvkov je kľúčová pre korektnosť simulácie, pretože umožňuje verne modelovať náhodné usporiadanie čísel v krabiciach a zachovať platnosť teoretických pravdepodobností.

2.2.3 Realizácia cyklickej stratégie

Jadrom algoritmu predstavuje metóda simulujSExistujucimiKrabicami, ktorá pre každého väzňa (vezenId) začína hľadanie v krabici s indexom zodpovedajúcim jeho identifikačnému číslu. Logika rozhodovania je implementovaná nasledovne:

Logika rozhodovania je nasledovná:

1. **Overenie identity:** Ak hodnota v krabici súhlasí s ID väzňa, pokus je úspešný a proces hľadania sa pre daného jednotlivca ukončí.
2. **Prechod na ďalší prvok:** Ak sa hodnota nezhoduje, algoritmus v ďalšom kroku pristúpi ku krabici s indexom, ktorý je určený touto načítanou hodnotou.
3. **Iterácia a limit:** Tento proces sa opakuje až do momentu nájdenia zhody alebo do vyčerpania limitu maxPokusov (štandardne 50 % celkového počtu krabíc).

Táto implementácia presne kopíruje princíp cyklov v náhodnej permutácii, čím je zachovaná matematická validita simulácie. Uvedený postup tak umožňuje exaktné empirické overenie teoretickej pravdepodobnosti úspechu.

2.2.4 Analýza štruktúry cyklov

Pre hlbšie pochopenie správania systému bola implementovaná metóda najdiVsetkyCykly(). Táto metóda využíva pole príznakov (boolean[] navstivene) na systematické prechádzanie celej permutácie, pričom každý prvok je navštívený práve raz.

Algoritmus funguje tak, že pre každý nenavštívený prvok začína sledovať jeho cyklus, zaznamenáva postupnosť prvkov až do návratu k počiatočnému bodu a následne označí všetky prvky v tomto cykle ako navštívene. Týmto spôsobom sa náhodná permutácia rozloží na disjunktné cykly, čo je kľúčové pre následnú štatistickú analýzu úspešnosti stratégie.

Táto funkcionalita umožňuje nielen konštatovať úspech alebo zlyhanie skupiny väzňov, ale aj exaktne identifikovať prítomnosť cyklov s dĺžkou $L > 50$, ktoré určujú kritický bod pre úspech stratégie. Dáta získané touto metódou tvoria základ pre praktickú časť práce, kde sa analyzuje distribúcia dĺžok cyklov a porovnáva sa s teoretickými predpokladmi vyplývajúcimi z kombinatoriky a teórie pravdepodobnosti.

2.3 Náhodná stratégia

Pre porovnanie optimálnej stratégie založenej na sledovaní cyklov bola implementovaná aj trieda RandomStrategy, ktorá simuluje náhodný výber krabíc pre každého väzňa. Táto stratégia slúži ako referenčný bod na overenie výhody cyklickej stratégie a demonštruje, ako výrazne pravdepodobnosť úspechu klesá pri ignorovaní štruktúry permutácie.

Implementácia

Trieda implementuje rozhranie Strategy a definuje dve základné metódy:

1. nazovStrategie() : vracia názov stratégie, v tomto prípade „Náhodná stratégia“.
2. pocitaj(int pocetVaznov, int maxPokusov) : vykonáva simuláciu pre zadaný počet väzňov a maximálny počet pokusov, a vracia počet úspešných väzňov.

Algoritmus

- **Inicializácia krabíc:** Najprv sa vytvorí zoznam krabíc reprezentovaných číslami od 0 po pocetVaznov – 1. Následne sa tento zoznam zamieša pomocou Collections.shuffle(), čím sa dosiahne náhodná permutácia čísiel v krabiciach.
- **Simulácia väzňov**: Pre každého väzňa sa vytvorí samostatný zoznam náhodne zamiešaných indexov krabíc, ktoré väzeň postupne skúša až do limitu maxPokusov. Ak väzeň nájde svoju krabicu, počet úspešných väzňov sa zvýši o 1 a pokračuje sa ďalším väzňom.
- **Vyhodnotenie výsledku:** Po skončení cyklov pre všetkých väzňov metóda vráti celkový počet úspešných väzňov. Tento algoritmus simuluje situáciu, kde väzni nevyužívajú žiadnu štruktúru permutácie a otvárajú krabice úplne náhodne.

Pravdepodobnosť úspechu celej skupiny pri náhodnej stratégii klesá exponenciálne s rastom počtu väzňov, pretože každý väzeň skúša krabice nezávisle od ostatných. Pre 100 väzňov s limitom 50 pokusov je šanca na prežitie celej skupiny $0{,}5^{100}$, čo je hodnota, ktorá sa v praktických simuláciách javí ako nulová.

2.4 Stratégia párnych a neparných čísel

Okrem optimálnej a náhodnej stratégie boli implementované aj heuristické stratégie označené ako stratégie párnych (EvenStrategy) a nepárnych (OddStrategy) čísel. Ich cieľom je demonštrovať, že systematický, avšak matematicky nepodložený výber krabíc nevedie k výraznému zvýšeniu pravdepodobnosti úspechu.

Triedy EvenStrategy a OddStrategy implementujú rozhranie Strategy a definujú metódy nazovStrategie() a pocitaj(int pocetVaznov, int maxPokusov).

2.4.1 Stratégia párnych čísel

Algoritmus

1. **Inicializácia krabíc**: Podobne ako pri ostatných stratégiách sa najskôr vytvorí zoznam krabíc obsahujúci identifikátory väzňov od 0 po (pocetVaznov – 1). Následne sa použije metóda Collections.shuffle(), čím sa vytvorí náhodná permutácia reprezentujúca reálne rozmiestnenie čísel v krabiciach.
2. **Výber krabíc väzňov**: Každý väzeň postupne prechádza zoznam krabíc, avšak otvára iba krabice s párnym indexom (i % 2 == 0). Pri každom otvorení sa kontroluje, či hodnota v krabici zodpovedá jeho identifikačnému číslu:
    a. Ak áno, väzeň je označený ako úspešný.
    b. Ak nie, pokračuje sa na ďalšiu párnu krabicu.
3. **Obmedzenie počtu pokusov**: Počet otvorených krabíc je sledovaný pomocou premennej otvoreneKrabice. Ak väzeň dosiahne limit maxPokusov, proces sa pre neho ukončí, aj keď ešte neprešiel všetky párne indexy.

Charakteristika stratégie

Stratégia párnych čísel predstavuje deterministický, ale štrukturálne obmedzený prístup. Väzeň v tomto prípade ignoruje polovicu dostupných krabíc (nepárne indexy). Ak sa hľadaný identifikátor nachádza na nepárnej pozícii, väzeň ho pri tomto postupe nemôže nájsť, čím sa pravdepodobnosť úspechu jednotlivca fixuje na maximálne 50 %.

Z hľadiska analýzy táto stratégia:

- Nevyužíva štruktúru permutácie ani prepojenie medzi lístkom a indexom krabice.

- Demonštruje, že samotná systematickosť výberu (poradie) bez matematického základu nie je postačujúca.
- Slúži ako názorný príklad pre komparatívnu analýzu v praktickej časti práce, kde zvýrazňuje efektivitu optimálnej cyklickej stratégie.

2.4.2 Stratégia nepárnych čísel (OddStrategy)

Stratégia nepárnych čísel (OddStrategy) predstavuje alternatívny heuristický prístup, ktorý je koncepčne podobný stratégii párnych čísel. Jej cieľom je analyzovať správanie systému pri systematickom, avšak matematicky neoptimalizovanom výbere krabíc, čím dopĺňa komparatívnu analýzu k stratégii párnych čísel.

Postup algoritmu je realizovaný nasledovne:

1. **Inicializácia krabíc**: Systém vytvorí zoznam identifikátorov od 0 po (pocetVaznov – 1), ktorý reprezentuje obsah krabíc. Tento zoznam je následne náhodne premiešaný pomocou metódy Collections.shuffle(), čím sa vytvorí rovnomerná náhodná permutácia.
2. **Výber krabíc väzňom**: Každý väzeň postupne prechádza zoznam krabíc, pričom otvára iba krabice s nepárnym indexom (podmienka i % 2 != 0). Pri každom otvorení sa kontroluje, či hodnota v krabici zodpovedá identifikačnému číslu väzňa:
    a. Ak sa hodnota zhoduje, väzeň je označený ako úspečný.
    b. Ak nie, pokračuje sa na ďalší nepárny index.
3. **Limit pokusov**: Počet otvorených krabíc je obmedzený parametrom maxPokusov. Ak väzeň dosiahne tento limit, jeho pokus sa ukončí bez ohľadu na to, či ešte existujú ďalšie nepárne indexy.

Charakteristika stratégie

Stratégia nepárnych čísel je deterministická a výrazne obmedzuje priestor vyhľadávania tým, že systematicky ignoruje polovicu dostupných krabíc (tie s párnym indexom).

Podobne ako pri stratégii párnych čísel:

- Nevyužíva štruktúru cyklov v permutácii
- Neadaptuje sa na priebežné výsledky

- Nezvyšuje pravdepodobnosť úspechu oproti náhodnej stratégii významným spôsobom

Jej implementácia má experimentálny a didaktický charakter. Slúži na demonštráciu faktu, že samotná pravidelnosť výberu (napríklad podľa parity indexy) nevedie k efektívnemu riešeniu problému. Skutočný nárast pravdepodobnosti úspechu je dosiahnuteľný až využitím cyklickej štruktúry permutácie, ako je to v prípade optimálnej stratégie.

2.4.3 Stratégia spoločného vylúčenia (ExclusionStrategy)

Stratégia spoločného vylúčenia predstavuje experimentálny heuristický prístup založený na kolektívnom rozhodnutí skupiny väzňov vopred eliminovať časť prehľadávaného priestoru. Cieľom je analyzovať, či systematické zúženie počtu dostupných krabíc dokáže ovplyvniť celkovú úspešnosť simulácie v porovnaní s čistou náhodou.

Algoritmus hľadania je definovaný nasledovne:

1. **Inicializácia krabíc:** Podobne ako pri ostatných stratégiách, na začiatku sa vytvorí zoznam identifikátorov reprezentujúci obsah krabíc, ktorý je následne náhodne premiešaný pomocou metódy Collections.shuffle(), čím sa zabezpečí objektívny počiatočný stav.
2. **Definovanie zakázanej zóny:** Väzni sa pred začatím pokusu dohodnú na počte krabíc (parameter „pocetVylucenych“), ktoré nebudú otvorené žiadnym členom skupiny. Program vytvorí zoznam dostupných indexov, ktorý začína až od indexu zodpovedajúceho počtu vylúčených krabíc.
3. **Proces hľadania (náhodný výber):** Každý väzeň pristupuje k množine dostupných indexov individuálne a otvára krabice v náhodnom poradí. Podobne ako pri čistej náhodnej simulácií, aj tu väzeň otvára krabice v náhodnom poradí bez sledovania lístkov ako odkazov na ďalšie krabice. Rozdielom je len to, že jeho výberový priestor je vopred zmenšený o „zakázanú zónu“.
    a. **Úspech:** Ak nájdená hodnota zodpovedá ID väzňa, jeho pokus je úspešný
    b. **Neúspech:** V opačnom prípade väzeň pokračuje na ďalší dostupný index v rámci svojho poradia

4. **Dynamický limit pokusov:** Algoritmus vypočítava reálny limit ako minimum medzi povoleným počtom pokusov (štandardne 50) a skutočným počtom dostupných (nevylúčených) krabíc. Týmto sa predchádza chybám pri prístupe k indexom v prípade, ak by počet vylúčených krabíc bol príliš vysoký.

Charakteristika stratégie a logické zlyhanie stratégie

Stratégia spoločného vylúčenia je postavená na predpoklade, že zmenšenie výberového priestoru by mohlo zvýšiť šancu na nájdenie čísla v zostávajúcich krabiciach. Avšak z hľadiska teórie pravdepodobnosti táto stratégia:

- **Deterministické zlyhanie (Princíp úplnosti množiny):** Keďže počet väzňov aj počet krabíc je zhodný ($n = 100$) a každé identifikačné číslo sa v množine krabíc nachádza práve raz, vylúčenie akejkoľvek krabice z procesu hľadania znamená automatické zablokovanie prístupu k lístku konkrétneho väzňa. Pri vylúčení desiatich krabíc je matematicky garantované, že presne desať väzňov nemá žiadnu fyzickú možnosť nájsť svoje číslo, bez ohľadu na počet ich pokusov. Keďže podmienkou úspechu simulácie je nájdenie lístkov všetkými členmi skupiny, pravdepodobnosť prežitia klesá na nulu už v momente definovania "zakázanej zóny".
- **Neadaptívnosť procesu:** Výber krabíc prebieha náhodne (ako pri RandomStrategy), čo znamená, že väzni nevyužívajú štruktúru permutácie (cykly). Každý väzeň hľadá "na vlastnú päsť" v zmenšenom poli, čím sa šanca na kolektívny úspech stále správa ako súčin individuálnych pravdepodobností, čo pri 100 väzňoch vedie k prakticky nulovému výsledku.

Implementácia tejto stratégie v aplikácii slúži na didaktické účely. Demonštruje dôležitý fakt, že akékoľvek obmedzenie počtu krabíc bez nadväznosti na ich obsah (na rozdiel od optimálnej stratégie sledovania cyklov) nevedie k zlepšeniu výsledkov, ale vnáša do systému nevyhnutné body zlyhania.

2.4.4 Hybridná stratégia (HybridStrategy)

Hybridná stratégia predstavuje porovnávací model navrhnutý na skúmanie vplyvu narušenia kolektívnej koordinácie. Táto stratégia kombinuje dva protichodné prístupy – optimálnu cyklickú metódu a neefektívny náhodný výber – v rámci jednej skupiny väzňov v tom istom simulačnom prostredí.

Algoritmus hľadania je definovaný nasledovne:

1. Inicializácia systému: Podobne ako pri predchádzajúcich modeloch, na začiatku sa generuje jednotná sada krabíc obsahujúca náhodne premiešané lístky s identifikátormi väzňov (0 až n - 1). Táto konfigurácia je statická pre všetkých členov skupiny v rámci jedného simulačného behu.
2. Segmentácia skupiny: Väzni sú rozdelení do dvoch rovnocenných podskupín podľa ich poradového čísla:
    a. Prvá podskupina (Indexy 0 - 49): Aplikuje cyklickú stratégiu (Cycle Strategy). Každý väzeň začína na krabici so svojim číslom a následne sa riadi hodnotou nájdeného lístka ako smerovníkom na ďalšiu krabicu.
    b. Druhá podskupina (Indexy 50 - 99): Aplikuje stratégiu náhodného výberu (Random Strategy). Každý väzeň si generuje vlastnú unikátnu permutáciu indexov krabíc, ktoré následne otvára bez ohľadu na ich obsah.
3. Proces verifikácie úspechu:
    a. Individuálny úspech: Väzeň je úspešný, ak v rámci stanoveného limitu (štandardne 50 pokusov) nájde krabicu obsahujúcu jeho ID.
    b. Kolektívny úspech: Celá simulácia končí úspechom len vtedy, ak podmienku individuálneho úspechu splní všetkých 100 väzňov (obe podskupiny).

Charakteristika stratégie a logické zlyhanie

Hybridná stratégia slúži ako významný analytický nástroj na demonštráciu krehkosti optimálnych riešení. Hoci prvá polovica skupiny maximalizuje svoje šance využitím matematickej štruktúry cyklov, celkový výsledok je znehodnotený druhou skupinou z nasledujúcich dôvodov:

- Súčin nezávislých pravdepodobnosti: Zatiaľ čo pri cyklickej stratégii je úspech väzňov silne korelovaný (závisí od dĺžky najdlhšieho cyklu v permutácii), pri náhodnej podskupine sú úspechy jednotlivcov nezávislé udalosti. Pravdepodobnosť, že všetkých 50 väzňov v náhodnej skupine uspeje, je vyjadrená ako $\left(\frac{1}{2}\right)^{50}$. Táto hodnota je takmer nulová ($\approx 8{,}88 \cdot 10^{-16}$), čo v praxi znamená, že náhodná skupina takmer s istotou zlyhá bez ohľadu na úspech cyklickej skupiny.
- Kritická závislosť na najslabšom článku: Simulácia potvrdzuje, že prežitie skupiny nie je určené priemernou úspešnosťou, ale pravdepodobnosť úspechu najmenej efektívnej zložky. Aj keby cyklická skupina dosiahla teoretické maximum úspešnosti (≈ 31 %), hybridný systém ako celok vykazuje nulovú šancu na prežitie.

Implementácia tejto stratégie má v aplikácii edukačný charakter. Umožňuje používateľovi vizuálne porovnať dva odlišné stochastické procesy prebiehajúce nad rovnakou dátovou sadou a pochopiť, prečo je pre úspech v hádanke o 100 väzňoch nevyhnutná stopercentná metodická disciplína celej skupiny.

2.5 Architektúra a objektový návrh aplikácie

Aplikácia je navrhnutá s dôrazom na modularitu a znovupoužiteľnosť kódu, pričom využíva overené princípy objektovo orientovaného programovania (OOP) a architektonický vzor MVC (Model-View-Controller). Tento prístup zabezpečuje striktné oddelenie stavu aplikácie a simulačnej logiky od prezentačnej vrstvy, čo je kľúčové pre stabilitu a testovateľnosť systému.

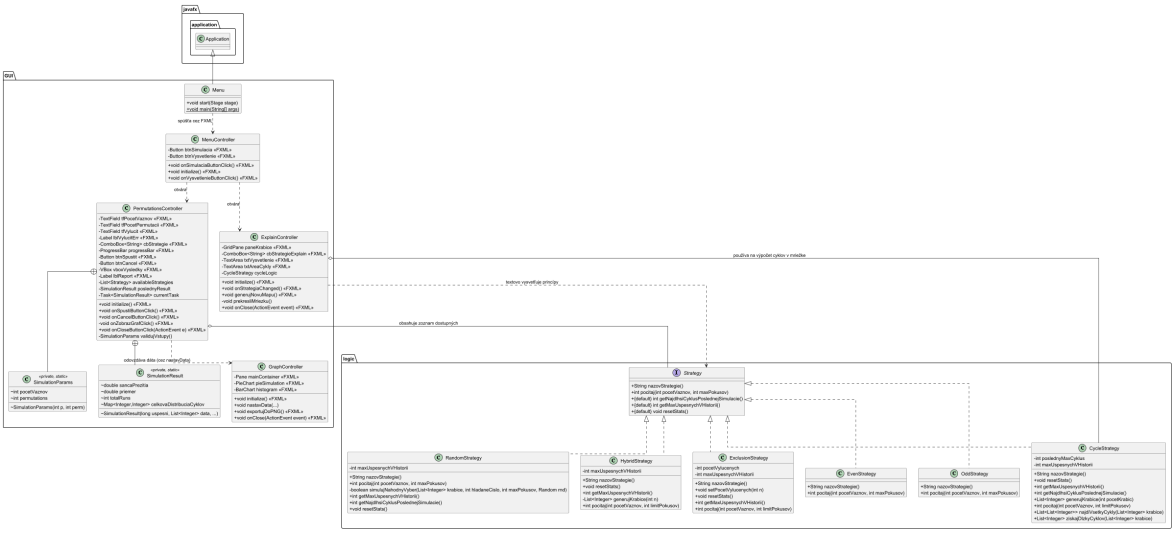
Hlavné výhody tohto prístupu sú:

- **Modularita a abstrakcia**: Použitie rozhraní (napr. interface Strategy) umožňuje definovať všeobecný kontrakt pre výpočet úspešnosti, pričom konkrétne algoritmy sú zapuzdrené vo vlastných triedach.
- **Škálovateľnosť**: Vďaka architektúre MVC je možné pridať nové typy vizualizácií (napr. dodatočné typy grafov) alebo nové simulačné stratégie bez nutnosti modifikovať existujúcu logiku riadenia.

- **Udržiavateľnosť**: Jasné rozdelenie zodpovedností uľahčuje lokalizáciu prípadných chýb a zjednodušuje proces rozširovania aplikácie o nové funkcie (napr. export dát do formátu PNG).

2.5.1 Hierarchia balíkov a komponentov

Základná štruktúra projektu je rozdelená do logických modulov, ktoré striktne reflektujú architektúru MVC a zabezpečujú prehľadnú organizáciu zdrojového kódu. Celková hierarchia tried, ich atribútov a vzájomných väzieb je znázornená na diagrame tried nižšie (Obrázok 4).:



Obrázok 4 UML diagram tried

- **Balík logic (Model)**: Predstavuje výpočtové jadro aplikácie. Obsahuje implementácie algoritmov definujúcich správanie väzňov, pričom všetky stratégie implementujú spoločné rozhranie Strategy. To zaručuje polymorfizmus a jednotný spôsob volania simulácie. Patria sem:
    - **CycleStrategy**: Optimálna stratégia založená na matematickej teórii sledovaní cyklov v permutácii.
    - **RandomStrategy** : Referenčný model náhodného výberu slúžiaci na komparatívnu analýzu.
    - **EvenStrategy a OddStrategy:** Heuristické modely zamerané na výber krabíc s konkrétnou paritou.
    - **ExclusionStrategy:** Experimentálny prístup založený na kolektívnom zúžení výberového priestoru.

- **HybridStrategy:** Kombinovaný model integrujúci prvky cyklickej a náhodnej stratégie pre skúmanie vplyvu narušenia koordinácie.

- **Balík GUI (Controller)**: Obsahuje triedy riadiace tok dát medzi používateľom a logikou. Tieto triedy reagujú na udalosti (event-driven programming) a zabezpečujú.
  - **MenuController:** Predstavuje hlavný vstupný bod pre konfiguráciu simulácie.
    - **Validácia vstupov:** Implementuje logiku pre overenie korektnosti používateľských dát (kontrola nezáporných hodnôt, celočíselných rozsahov a limitov pre počet väzňov).
    - **Správa vlákien:** Na spustenie simulácií využíva JavaFX Task a Service, čím zabezpečuje, že náročné výpočty prebiehajú na pozadí a nespôsobujú zamrznutie používateľského rozhrania.
    - **Riadenie stavu UI:** Dynamicky prepína dostupnosť ovládacích prvkov v závislosti od stavu simulácie (napr. blokovanie zmien počas behu výpočtu).
  - **GraphController:** Zabezpečuje transformáciu surových štatistických dát do vizuálnej podoby.
    - **Spracovanie modelov:** Prijíma predvypočítané dátové modely (distribúcia úspechov, dĺžky cyklov) a mapuje ich na prvky BarChart a PieChart.
    - **Optimalizácia zobrazenia:** Implementuje ScrollPane pre histogramy, čím predchádza vizuálnej degradácii (stláčaniu stĺpcov) pri veľkom počte kategórií.
    - **Podmienené vykresľovanie:** Obsahuje logiku pre selektívne zobrazenie analýzy cyklov, ktorá sa aktivuje primárne pri voľbe cyklickej stratégie.
  - **PermutationsController:** Špecializovaný nástroj pre mikroanalýzu simulácie.
    - **Krokovanie cyklov:** Umožňuje používateľovi prechádzať permutáciu krok po kroku, pričom vizuálne zvýrazňuje aktuálne prehľadávanú krabicu a štruktúru cyklu.

        - **Synchronizácia s UI:** Využíva Platform.runLater() na bezpečné aktualizovanie grafických prvkov z výpočtového vlákna späť do hlavného aplikačného vlákna.
    - **ExplainController:** Zabezpečuje edukačnú vrstvu aplikácie. Podporuje navigáciu v rámci teoretického obsahu a vizualizuje pravidlá hádanky, čím zvyšuje pridanú hodnotu aplikácie pre koncového používateľa.
    - **Menu (Entry Point):** Štartovacia trieda dediacia od javafx.application.Application. Zodpovedá za inicializáciu aplikačného kontextu, nastavenie globálneho spracovania chýb (error handling) a logovanie systémových udalostí.
- **Adresár resources (View)**: Obsahuje deklaratívne súbory vo formáte .fxml. Táto vrstva definuje vizuálnu topológiu používateľského rozhrania prostredníctvom technológie JavaFX. Oddelenie dizajnu od programového kódu umožňuje rýchlu modifikáciu grafických prvkov (napr. úprava štýlov pomocou CSS) bez nutnosti rekompilácie simulačnej logiky.

2.5.2 Interakcia medzi komponentmi

Interakcia medzi jednotlivými vrstvami architektúry MVC je navrhnutá tak, aby maximalizovala separáciu zodpovedností a zabezpečila odozvu aplikácie aj pri extrémnej výpočtovej záťaži. Kľúčovým prvkom je Model, situovaný v balíku logic, ktorý vystupuje ako pasívna a na používateľskom rozhraní úplne nezávislá zložka (tzv. UI-agnostic). Táto izolácia je kritická nielen pre čistotu návrhu, ale aj pre možnosť nezávislého testovania simulačných algoritmov bez nutnosti spúšťať grafické prostredie. Model striktne dbá na dátovú integritu tým, že validuje prijaté konfigurácie a v prípade neplatných stavov vyhadzuje jasne definované výnimky. Výsledky simulácií, ako sú distribúcie úspechov či analýzy cyklov, neposkytuje formou priameho prístupu k interným stavom, ale transformuje ich do nemenných dátových štruktúr alebo špecializovaných prepravných objektov (DTO - Data Transfer Objects). Týmto mechanizmom sa predchádza nechceným vedľajším účinkom pri následnej manipulácii s dátami v prezentačnej vrstve.

Prezentačná vrstva, reprezentovaná adresárom resources (View), definuje statickú vizuálnu topológiu pomocou súborov FXML. Prvky rozhrania sú s riadiacou logikou prepojené deklaratívne prostredníctvom identifikátorov fx:id, pričom vizualizačné kontajnery sú navrhnuté ako responzívne a skrolovateľné. Táto technická voľba je

nevyhnutná pre zachovanie čitateľnosti histogramov pri vysokom počte kategórií, kedy by bez vertikálneho či horizontálneho posunu dochádzalo k vizuálnej degradácii dátových stĺpcov.

Úlohu aktívneho koordinátora plní Controller (balík GUI), ktorého procesná logika prebieha v troch cyklických fázach. V prvej fáze dochádza k zberu a validácii vstupov, kde kontrolér overuje napríklad nezápornosť a celočíselnosť parametrov simulácie. Následne prechádza do fázy asynchrónneho riadenia, kedy výpočtové úlohy deleguje na pozadie prostredníctvom tried JavaFX Task alebo Service. Týmto oddelením sa odľahčí hlavné vlákno aplikácie (UI Thread), čím sa eliminuje riziko zamŕzania rozhrania počas miliónov simulačných iterácií. Počas behu výpočtu kontrolér preventívne deaktivuje ovládacie prvky, aby zamedzil konfliktným vstupom. V záverečnej fáze Controller pomocou bezpečných spätných volaní a metódy Platform.runLater() extrahuje štatistické ukazovatele z Modelu a reaktívne aktualizuje vizuálne komponenty vo View. Tento prístup zabezpečuje, že používateľ má okamžitú odozvu o stave systému pri zachovaní stability celej aplikácie.

2.5.3 Správa verzií a archivácia zdrojového kódu

Pre efektívny vývoj aplikácie a zabezpečenie integrity zdrojových súborov bol počas celého procesu implementácie využívaný distribuovaný systém správy verzií Git. Tento nástroj umožňoval detailné sledovanie zmien v kóde, bezpečný návrat k predchádzajúcim funkčným verziám a systematické vetvenie projektu pri experimentovaní s novými simulačnými stratégiami. Použitie systému Git vytvorilo presnú historickú stopu vývoja vo forme záznamov (tzv. commits), ktoré dokumentujú progres práce a logickú nadväznosť jednotlivých fáz projektu.

Celý zdrojový kód bol priebežne synchronizovaný a archivovaný vo vzdialenom repozitári na platforme GitHub. Táto hostingová služba rozširuje možnosti systému Git o pokročilé nástroje na správu projektov a garantuje bezpečnú zálohu kódu na cloudových serveroch, čím sa minimalizuje riziko straty dát v prípade lokálneho zlyhania hardvéru. Využitie platformy GitHub zároveň zvyšuje transparentnosť práce, nakoľko umožňuje školiteľovi či oponentovi jednoduchý prístup k aktuálnej verzii projektu, dokumentácii a histórii úprav prostredníctvom prístupového odkazu. Tento prístup k správe verzií je v súlade s modernými štandardmi vývoja softvéru a zabezpečuje plnú reprodukovateľnosť výsledkov simulácie.

2.6 Interaktívne grafické používateľské rozhranie

V súlade so zadaním práce bolo navrhnuté interaktívne prostredie aplikácie, ktoré slúži ako komplexný nástroj na experimentálne overovanie teoretických hypotéz. Pri návrhu bol kladený dôraz na modularitu a responzivitu, čo umožňuje plynulé pracovanie veľkého objemu dát (milióny replikácií) bez negatívneho vplyvu na používateľskú skúsenosť.

2.6.1 Moduly aplikácie

Pre lepšiu orientáciu používateľa je rozhranie rozdelené na špecifické pohľady:

- **Menu**: Slúži ako hlavný rozcestník aplikácie. Umožňuje používateľovi navigáciu medzi jednotlivými modulmi, konfiguráciu vstupných parametrov simulácie a voľbu testovanej stratégie (triedy Menu a MenuController).
- **PermutationsView**: Tento modul sa zameriava na analýzu dát. Umožňuje vizualizáciu konkrétnych náhodných permutácií, interaktívnu prácu s krabicami a sledovanie správania jednotlivých väzňov v reálnom čase (spravuje PermutationsController).
- **GraphView**: Zabezpečuje analýzu a grafickú prezentáciu štatistických výsledkov. Umožňuje porovnávanie úspešnosti rôznych stratégií prostredníctvom dynamických grafov a analýzu distribúcie dĺžok cyklov (spravuje GraphController).
- **ExplainView**: Plní edukačnú funkciu tým, že poskytuje teoretické pozadie problému. Obsahuje vysvetlenie matematických princípov stratégie sledovania cyklov a intuitívne komentáre k interpretácii výsledkov (spravuje ExplainController).

2.6.2 Algoritmická logika pohľadov (View)

Grafické pohľady aplikácie neplnia iba statickú zobrazovaciu funkciu. Prostredníctvom úzkej väzby na triedy typu Controller zabezpečujú dynamické riadenie celého životného cyklu simulácie – od prvotného spracovania a validácie používateľských vstupov až po reaktívnu aktualizáciu výsledkov v reálnom čase. Táto vrstva zodpovedá za transformáciu abstraktných matematických dát z modelu do formy, ktorá je pre používateľa vizuálne zrozumiteľná.

2.6.3 Permutations View – používateľské rozhranie pre simuláciu

Trieda PermutationsController spolu s FXML súborom predstavuje samostatné okno aplikácie určené na nastavenie a spustenie simulácie problému väzňov. Ide o formulárové rozhranie, ktoré umožňuje používateľovi definovať vstupné parametre experimentu a následne zobraziť štatistické vyhodnotenie výsledkov.

Grafická štruktúra rozhrania

Používateľské rozhranie je postavené na technológii JavaFX, pričom jeho vizuálna hierarchia je deklarovaná v súbore FXML. Tento prístup umožňuje striktné oddelenie grafického návrhu od obslužnej logiky. Hlavné ovládacie prvky rozhrania zahŕňajú:

- **Textové polia (TextField):** Slúžia na parametrizáciu experimentu, konkrétne na zadanie počtu väzňov a definovanie celkového počtu simulačných replikácií.
- **Rozbaľovací zoznam (ComboBox):** Komponent na intuitívny výber testovanej stratégie zo zoznamu dostupných algoritmov.
- **Ovládacie tlačidlá (Button):** Súbor prvkov na spustenie výpočtového procesu, prerušenie simulácie alebo bezpečné zatvorenie okna.

Rozhranie je navrhnuté ako modálne okno, čím je vizuálne aj funkčne oddelené od hlavného menu. Tento dizajn minimalizuje kognitívnu záťaž používateľa a jasne definuje postupnosť krokov potrebných na vykonanie experimentu..

Inicializácia a dynamické načítanie stratégií

Proces spustenia modulu riadi metóda initialize(), ktorá pripravuje prostredie pre beh simulácie v dvoch kľúčových krokoch:

1. Konfigurácia predvoleného stavu: Systém automaticky nastavuje štandardné parametre (100 väzňov a 10 000 simulácií), ktoré reprezentujú najbežnejšiu testovaciu vzorku v rámci teoretického problému.
2. Dynamická registrácia stratégií: Kontrolér inicializuje zoznam dostupných algoritmov, ktorý zahŕňa:
    a. CycleStrategy, RandomStrategy, HybridStrategy, EvenStrategy, OddStrategy a ExclusionStrategy.

Každá z týchto tried implementuje spoločné rozhranie Strategy, čo predstavuje kľúčový aspekt polymorfizmu v aplikácii. Do rozbaľovacieho zoznamu sa neukladajú priame referencie na objekty, ale textové reťazce získané metódou nazovStrategie(). Toto riešenie zabezpečuje vysokú mieru škálovateľnosti: pridanie novej stratégie si nevyžaduje žiadne zásahy do riadiacej logiky kontroléra, stačí vytvoriť novú triedu implementujúcu dohodnutý kontrakt rozhrania.

Mechanizmy validácie vstupov a ošetrenia vstupov

Robustnosť aplikácie je zaistená striktnou validáciou používateľských vstupov ešte pred inicializáciou samotného výpočtového jadra. Tento proces prebieha v dvoch rovinách:

- **Syntaktická validácia:** Kontrola, či zadané reťazce v textových poliach predstavujú platné celočíselné hodnoty.
- **Sémantická validácia:** Overenie, či sú zadané parametre (počet väzňov, počet simulácií) kladné čísla väčšie ako nula.

V prípade detekcie neplatného vstupu aplikácia preruší spracovanie a prostredníctvom modálneho okna Alert typu ERROR informuje používateľa o chybe. Tento mechanizmus efektívne predchádza výnimkám typu NumberFormatException a zaručuje stabilitu programu aj pri nepredvídateľnom správaní používateľa.

Asynchrónna implementácia simulačného jadra

Vzhľadom na vysokú výpočtovú náročnosť pri vykonávaní miliónov replikácií je jadro simulácie implementované asynchrónne s využitím triedy Task z knižnice JavaFX. Tento prístup je kritický pre zachovanie responzivity hlavného vlákna aplikácie (UI Thread), ktoré zostáva voľné pre prekresľovanie grafických prvkov a obsluhu používateľských interakcií.

Proces asynchrónneho výpočtu prebieha v nasledujúcich fázach:

1. **Príprava experimentu:** Na základe konfigurácie sa určí limit pokusov (štandardne $\frac{n}{2}$) a prostredníctvom rozhrania Strategy sa vyberie príslušná inštancia algoritmu.
2. **Iteratívny proces replikácií:** V rámci dedikovaného výpočtového vlákna prebieha cyklus simulácií. Pre každú iteráciu sa generuje unikátna náhodná permutácia, nad ktorou zvolená stratégia vyhodnocuje úspešnosť každého väzňa.

3. **Zber a agregácia surových dát:** Počet úspešných väzňov z každej simulácie sa ukladá do dynamickej štruktúry List<Integer>. Tieto surové dáta tvoria základ pre následnú štatistickú analýzu a konštrukciu histogramov v module GraphView.
4. **Monitorovanie a kontrola:** Aplikácia v reálnom čase komunikuje s používateľom prostredníctvom komponentu ProgressBar, ktorý vizualizuje progres úlohy. Používateľ má zároveň možnosť prebiehajúci výpočet kedykoľvek bezpečne ukončiť volaním metódy task.cancel(), čím je zabezpečená plná kontrola nad systémovými prostriedkami..

Deskriptívna štatistika a prezentácia výsledkov

Po úspešnom dokončení celého cyklu replikácií preberá agregované surové dáta GraphController, ktorý nad nimi vykonáva hĺbkovú deskriptívnu analýzu. Výstupy tejto analýzy sú následne prezentované prostredníctvom nasledujúcich metrík:

- **Pravdepodobnosť prežitia skupiny:** Kľúčový ukazovateľ vyjadrujúci percentuálny pomer simulácií, v ktorých uspeli všetci väzni (individuálna úspešnosť 100 %). Tento údaj slúži na priamu konfrontáciu empirických dát s teoretickým predpokladom danej stratégie (napr. ≈ 31 % pri stratégii cyklov).
- **Analýza stredných hodnôt:** Zahŕňa výpočet aritmetického priemeru a mediánu počtu úspešných väzňov v rámci jednej simulácie. Tieto metriky umožňujú posúdiť typické správanie systému a stabilitu zvolenej stratégie.
- **Miera variability:** Výpočet smerodajnej odchýlky, ktorá indikuje rozptyl nameraných výsledkov okolo priemeru. Vysoká variabilita poukazuje na nestabilitu stratégie a vyššiu mieru rizika celkového zlyhania skupiny.
- **Extrémne hodnoty:** Systém monitoruje rekordný počet úspešných väzňov naprieč všetkými simuláciami. Dosiahnutie absolútneho úspechu (záchrana celej skupiny) je v štatistickom paneli vizuálne signalizované zeleným zvýraznením, čo zvyšuje intuitívnosť rozhrania.

Finálne štatistické ukazovatele sú doplnené o interaktívny histogram (distribúcia početnosti úspechov) a koláčové grafy (pomer globálnych úspechov a zlyhaní). Táto kombinácia poskytuje používateľovi komplexný obraz o stochastických vlastnostiach

testovanej stratégie a umožňuje hlbšie pochopenie pravdepodobnostných zákonitostí problému.

2.6.4 Kontrola priebehu a prerušenie simulácie

Vzhľadom na vysokú výpočtovú náročnosť pri miliónových replikáciách bol do rozhrania integrovaný mechanizmus pre dynamickú kontrolu nad bežiacim procesom. Tieto prvky zvyšujú používateľský komfort a zabezpečujú stabilitu systému:

- **Indikátor progresu (Progress Bar):** Poskytuje používateľovi okamžitú vizuálnu spätnú väzbu o stave výpočtu. Tento komponent je synchronizovaný s asynchrónnym jadrom simulácie, čo zamedzuje dojmu, že aplikácia nereaguje (tzv. "freezing"), a umožňuje odhadnúť zostávajúci čas operácie.
- **Tlačidlo na zastavenie simulácie (Stop):** Významným funkčným prvkom je možnosť kedykoľvek prerušiť prebiehajúcu simuláciu. Ak používateľ zistí, že zvolil nevhodné parametre alebo príliš vysoký počet replikácií, môže výpočet okamžite a bezpečne terminovať.
- **Technická implementácia:** Funkcia využíva mechanizmus prerušenia asynchrónneho vlákna (Background Thread). Po vyvolaní prerušenia systém bezpečne zastaví iteračné cykly, uvoľní alokované systémové prostriedky a vráti rozhranie do pôvodného stavu. Táto implementácia dôsledne predchádza kritickým chybám, akými sú pády aplikácie alebo úniky pamäte (Memory leaks).

2.6.5 Graph View - grafické zobrazenie výsledkov simulácie

Modul pre grafické zobrazenie výsledkov simulácie nepredstavuje len prostý zobrazovač číselných hodnôt, ale komplexný analytický celok. Z hľadiska softvérového inžinierstva sme implementáciu postavili na architektonickom vzore MVC (Model-View-Controller). Toto oddelenie logiky od prezentácie zabezpečuje vysokú modularitu kódu, kde deklaratívny opis rozhrania v súbore FXML úzko spolupracuje s riadiacou logikou v triede GraphController.

Architektonické riešenie a responzivita

Základom používateľského rozhrania je kontajner BorderPane, ktorý zabezpečuje logické a predvídateľné rozdelenie prvkov. Hlavný vizuálny obsah sme sústredili do komponentu VBox (identifikátor mainContainer), ktorý využíva moderné vizuálne

parametre vrátane jemného tieňovania a zaoblených rohov (definovaných prostredníctvom CSS vlastnosti -fx-background-radius: 15).

- **Dynamické škálovanie a binding:** Pre zabezpečenie korektného zobrazenia na rôznych rozlíšeniach monitorov sme využili mechanizmus DoubleBinding. Veľkosť grafických komponentov sa v reálnom čase prispôsobuje rozmerom okna, čím eliminujeme vizuálne artefakty a zachovávame čitateľnosť štatistických údajov bez ohľadu na mieru zväčšenia aplikácie.
- **Sématická farebná schéma:** Pre okamžitú kognitívnu orientáciu používateľa sme v systéme definovali fixné farebné konštanty. Zelená farba je striktne asociovaná s úspechom skupiny, červená reprezentuje neúspech a oranžová paleta slúži na vizualizáciu distribučných stĺpcov v histograme.

Komparatívna analýza pomocou koláčových grafov

Horná časť rozhrania integruje dva komponenty PieChart, ktoré umožňujú priame porovnanie empirických a teoretických dát:

- **Simulačný graf:** Zobrazuje reálne nameraný pomer úspešných a neúspešných pokusov z aktuálneho behu Monte Carlo simulácie.
- **Teoretický graf:** Zobrazuje matematický ideál (napr. 31,18 % pre cyklickú stratégiu). Táto paralela slúži na rýchlu verifikáciu správnosti algoritmu používateľom.
- **Customizácia vzhľadu:** Farby jednotlivých výsekov a legendy sú nastavované dynamicky po vytvorení grafických uzlov (Node). Týmto prístupom sme úspešne obišli obmedzenia štandardných JavaFX tém a zabezpečili sme vizuálnu konzistenciu naprieč celou aplikáciou.

Štatistický modul a Histogram

Dolná časť okna prináša hlbší pohľad na dáta prostredníctvom dvoch kľúčových prvkov:

- **Interaktívny BarChart (Histogram):** Zobrazuje frekvenciu výskyty počas úspešných väzňov.

- **Zoskupovanie dát (Binning):** Pri vysokom počte väzňov (n > 50) controller automaticky prepína na režim zoskupenia. Dáta sú pomocou TreeMap zoskupované do 50 intervalov (binov), čo zabraňuje vizuálnemu zahlteniu grafu a umožňuje sledovať tvar pravdepodobnostného rozdelenia.
- **Interaktivita:** Každý stĺpec histogramu je vybavený Tooltipom, ktorý sa zobrazí po 100 ms a obsahuje presné informácie o danom intervale. Implementované sú aj onMouseEntered eventy, ktoré menia jas stĺpca a kurzor myši, čím zvyšuje užívateľský komfort (UX).

- **Informačný panel štatistík:** Na výpočet kľúčových ukazovateľov v reálnom čase sme využili Java Streams API. Aplikácia tak efektívne generuje a zobrazuje aritmetický priemer, medián, smerodajnú odchýlku a maximálnu dosiahnutú hodnotu úspechu priamo počas priebehu simulácie.

Analýza distribúcie dĺžok cyklov (Špecifický historgram)

Okrem analýzy úspešnosti sme do aplikácie implementovali aj špecializovaný histogram zameraný na distribúciu dĺžok cyklov v rámci generovaných permutácií. Tento graf predstavuje kľúčový diagnostický nástroj na pochopenie vnútornej štruktúry problému:

- **Vizuálne potvrdenie kritickej hranice:** Histogram zobrazuje frekvenciu výskytu cyklov pre každú dĺžku od 1 do *n*. Používateľ tak môže na vlastné oči pozorovať, že kým krátke cykly sú v permutáciách prítomné takmer vždy, cykly s dĺžkou nad 50 sa vyskytujú s výrazne nižšou, ale stále podstatnou pravdepodobnosťou (približne 69 %).
- **Identifikácia príčiny zlyhania:** Tento graf priamo ilustruje matematický fakt, že v jednej permutácii môže existovať maximálne jeden cyklus dlhší ako 50. ak sa v grafe objaví hodnota v tomto „zakázanom“ intervale, pre používateľa je to okamžitý vizuálny signál, že simulácia pre danú konfiguráciu skončila neúspechom skupiny.
- **Štatistická stabilita:** Pri vysokom počte simulácií (napr. 10 000 a viac) tento histogram nadobúda charakteristický tvar, ktorý potvrdzuje teoretické rozdelenie dĺžok cyklov v náhodných permutáciách. To slúži ako ďalšia úroveň verifikácie nášho generátora náhodných čísiel

Export výsledkov a práce s grafickým kontextom

Významnou funkciou je tlačidlo „Exportovať PNG“, ktoré realizuje metóda exportujDoPNG().

- **Snapshot technológia:** Aplikácia vytvorí rastrovú kópiu celého kontajnera mainContainer pomocou metódy snapshot().
- **Práca s obrázkami:** Keďže štandardné knižnice môžu mať obmedzenia, implementovali sme vlastnú metódu fromFxImage(), ktorá využíva PixelReader na manuálnu transformáciu WritableImage na BufferedImage. Tento prístup zabezpečuje vysokú kompatibilitu pri ukladaní cez ImageIO.
- **Užívateľská spätná väzba:** Po úspešnom exporte sa zobrazí moderné dialógové okno s možnosťou priameho otvorenia priečinka alebo skopírovanie cesty k súboru do schránky (Clipboard).

2.6.6 Explain View – interaktívne vysvetlenie stratégií

Modul Explain View predstavuje edukačnú vrstvu aplikácie, ktorej primárnym cieľom je vizuálne a teoreticky objasniť mechanizmy jednotlivých stratégií. Na rozdiel od simulačného jadra sa táto časť nezameriava na hromadné štatistické výpočty, ale na demonštráciu logiky riešenia na konkrétnej inštancii problému. Implementácia je založená na deklaratívnom popise rozhrania v súbore FXML a riadiacej triede ExplainController.

Štruktúra používateľského rozhrania

Rozhranie sme rozdelili pomocou kontajnera HBox na dva funkčné celky, ktoré zabezpečujú prepojenie teórie s vizualizáciou:

1. **Analytická časť (ľavý panel):** Slúži na konfiguráciu a textovú interpretáciu. Obsahuje:
    a. ComboBox pre výber konkrétnej stratégie.
    b. Textové pole s podrobným teoretickým vysvetlením zvoleného postupu.
    c. Informačnú oblasť zobrazujúcu dekompozíciu aktuálnej permutácie na cykly.
    d. Tlačidlo na generovanie novej náhodnej permutácie.
2. **Vizuálne časť (pravý panel):** Poskytuje grafickú odozvu na zvolené parametre. Obsahuje:

a. Stavový indikátor úspechu alebo neúspechu skupiny pri danom rozložení.
b. Interaktívnu mriežku 100 krabíc (rozloženie 10 x 10).
c. Informačný riadok s dynamickým výpisom obsahu krabice.
d. Tlačidlo na ukončenie modulu.

Každá krabica je v rámci mriežky reprezentovaná ako grafický objekt (obdĺžnik) s priradeným indexom, pričom vizuálny štýl sa mení v závislosti od interakcie používateľa.

Inicializácia a výber stratégie

Pri inicializácii modulu aplikácia automaticky definuje šesť základných scenárov správania väzňov, ktoré reprezentujú rôzne prístupy k riešeniu problému:

- **Cyklická stratégia** (optimálne riešenie založené na dekompozícii permutácie),
- **Náhodná stratégia** (slúži ako referenčný bod s minimálnou úspešnosťou),
- **Stratégia párnych čísel a Stratégia nepárnych čísel** (heuristiky založené na parite),
- **Stratégia vylúčenia** (špecifický prístup eliminácie krabíc),
- **Hybridná stratégia** (kombinácia viacerých rozhodovacích mechanizmov).

Mechanizmus reaktívnej zmeny rozhrania sme implementovali tak, aby pri každej zmene výberu v komponente ComboBox došlo k atomickej aktualizácii stavu aplikácie prostredníctvom nasledujúcich operácií:

1. **Aktualizácia obsahu:** Dynamicky sa mení text teoretického vysvetlenia podľa priradeného kľúča stratégie, čo zabezpečuje okamžitý kontext pre používateľa.
2. **Nová inštancia problému:** Systém vygeneruje novú náhodnú permutáciu čísel. Tento krok je kľúčový pre demonštráciu robustnosti (v prípade cyklickej stratégie) alebo vysokej pravdepodobnosti zlyhania (v prípade ostatných stratégií) na rôznych rozloženiach dát.
3. **Vizuálny a dátový reset:** Dochádza k prekresleniu mriežky krabíc do východiskového stavu a k prepočítaniu štatistických informácií o cykloch, ktoré sú následne zobrazené v textovej oblasti.

Generovanie a spracovanie permutácie

Jadrom interaktívneho vysvetlenia je metóda generujNovuMapu(), ktorá zabezpečuje konzistenciu medzi modelom dát a ich vizuálnou reprezentáciou. Proces prebieha v nasledujúcich krokoch:

1. **Purifikácia dát:** Systém najskôr odstráni všetky predchádzajúce záznamy a vizuálne stavy z mriežky.
2. **Generovanie inštancie:** Vygeneruje sa nová náhodná permutácia obsahujúca 100 unikátnych prvkov.
3. **Konfigurácia vizualizácie:** Podľa aktuálne zvolenej stratégie v ComboBoxe sa nastavia pravidlá pre vykreslenie mriežky.

Pri cyklickej stratégii sa využívajú špecializované metódy triedy CycleStrategy:

- **generujKrabice(100):** Zabezpečuje vytvorenie počiatočnej náhodnej permutácie.
- **najdiVsetkyCykly():** Vykonáva dekompozíciu permutácie a identifikuje všetky uzavreté cykly.

Kľúčovým analytickým prvkom je kontrola podmienky úspechu: ak algoritmus identifikuje cyklus s dĺžkou L > 50, skupina je označená ako neúspešná. Táto podmienka je v používateľskom rozhraní okamžite vizuálne indikovaná zmenou stavového indikátora.

Vizualizácia cyklov a paritných stratégií

Grafická interpretácia dát umožňuje používateľovi pochopiť abstraktné matematické javy prostredníctvom konkrétnych vizuálnych podnetov

Cyklická stratégia:

- **Farebné kódovanie:** Každému identifikovanému cyklu systém priradí unikátnu náhodne generovanú farbu.
- **Logické zoskupenie:** Všetky krabice patriace do identického cyklu sú vykreslené rovnakou farbou, čo umožňuje okamžitú vizuálnu identifikáciu štruktúry permutácie.
- **Textová analýza:** V textovej oblasti sa generuje štruktúrovaný zoznam všetkých cyklov vrátane ich presnej dĺžky.

Tento prístup umožňuje používateľovi pochopiť príčinnú súvislosť medzi dĺžkou najdlhšieho cyklu a celkovou úspešnosťou skupiny.

Paritné stratégie (Párne a nepárne čísla):

Tieto stratégie slúžia ako komparatívny kontrast k optimálnemu cyklickému riešeniu.

- Vizuálna hierarchia: Pri stratégii párnych čísel sú zvýraznené len krabice s párnym indexom, zatiaľ čo nepárne sú zosvetlené (polopriesvitné), čím sú vizuálne potlačené do úzadia (pri nepárnej stratégii je proces opačný).
- Edukačný význam: Demonštrujú, že jednoduché deterministické obmedzenie výberu (zúženie množiny na 50 % krabíc bez ohľadu na ich obsah) vedie k nulovej pravdepodobnosti prežitia skupiny ($P=0$).

Dynamická responzivita a interaktivita rozhrania

Pre zabezpečenie optimálneho používateľského zážitku sme implementovali pokročilé techniky dynamického prispôsobovania prvkov. Veľkosť jednotlivých krabíc v mriežke je programovo viazaná na rozmery kontajnera GridPane prostredníctvom mechanizmu DoubleBinding.

Tento prístup zabezpečuje:

- **Automatické škálovanie:** Geometria krabíc sa plynule prispôsobuje zmenám veľkosti okna aplikácie bez straty vizuálnej integrity.
- **Konzistentnú čitateľnosť:** Algoritmus upravuje veľkosť písma a grafických prvkov tak, aby zostali informácie čitateľné na rôznych typoch displejov.
- **Vysokú úroveň UX:** Responzívny dizajn zvyšuje profesionálny dojem z implementácie a uľahčuje prácu s aplikáciou v režime viacerých okien.

Mechanizmus interaktívnej explorácie

Interaktivita modulu je postavená na spätnej väzbe v reálnom čase. Každý objekt reprezentujúci krabicu reaguje na udalosť kliknutia myšou, čo aktivuje aktualizáciu stavového riadku s informáciou: „Krabica X obsahuje lístok Y“.

Tento funkčný prvok umožňuje používateľovi:

- **Manuálnu verifikáciu:** Používateľ môže fyzicky prejsť konkrétny cyklus a overiť si správnosť farebného kódovania.
- **Simuláciu postupu:** Možnosť simulovať individuálne kroky väzňa prehlbuje pochopenie logiky zvolenej stratégie.
- **Zážitkové učenie:** Prepojenie vizuálneho vnemu s manuálnou interakciou efektívne vysvetľuje abstraktné matematické pravidlá.

Architektonické zhodnotenie modulu

Explain View plní v rámci celého projektu kľúčovú pedagogickú a demonštračnú funkciu. Zatiaľ čo simulačné jadro aplikácie je optimalizované na výkon pri hromadných štatistických výpočtoch, tento modul sa zameriava na hĺbkovú analýzu jednej konkrétnej inštancie problému.

Z hľadiska softvérového inžinierstva prináša oddelenie tejto logiky do samostatného kontroléra (ExplainController) niekoľko benefitov:

- **Zvýšená modularita:** Jasné oddelenie štatistickej a edukačnej logiky znižuje komplexnosť kódu a uľahčuje údržbu projektu.
- **Prehľadnosť architektúry:** Striktne dodržiavanie vzoru MVC umožňuje nezávislý vývoj grafického rozhrania a riadiacej logiky.
- **Škálovateľnosť:** Štruktúra modulu je pripravená na budúce rozšírenia, ako sú napríklad krokované animácie prechodu cyklov alebo vizuálne porovnanie dvoch permutácií súčasne.

2.6.7 Menu View – hlavné navigačné rozhranie aplikácie

Modul Menu View predstavuje centrálny navigačný bod celého systému. Jeho primárnou úlohou je poskytnúť používateľovi intuitívne rozhranie pre výber medzi dvoma komplementárnymi funkčnými režimami aplikácie:

- **Simulačný modul:** Zameraný na hromadné spracovanie dát a štatistický výpočet pravdepodobnosti.
- **Edukačný modul (Explain View):** Zameraný na interaktívnu vizualizáciu a analýzu logiky jednotlivých stratégií.

Z hľadiska softvérovej architektúry je modul implementovaný kombináciou deklaratívneho FXML súboru a riadiacej triedy MenuController, ktorá zabezpečuje prepínanie scén a správu životného cyklu okien.

Grafická štruktúra a dizajn

Pri návrhu rozloženia prvkov sme kládli dôraz na vizuálnu rovnováhu a hierarchiu informácií. Základným kontajnerom je AnchorPane, ktorý v sebe integruje centrovaný StackPane. Toto vrstvenie zabezpečuje, že hlavný obsah zostáva fixne umiestnený v strede obrazovky bez ohľadu na zmenu veľkosti okna.

Centrálny panel (organizovaný pomocou VBox) obsahuje tri hierarchické úrovne:

- **Identifikačná zóna:** Dominuje jej názov aplikácie „100 Prisoners Problem“ doplnený o deskriptívny podnadpis, ktorý definuje zameranie softvéru.
- **Interaktívna zóna:** Obsahuje dve dominantné akčné tlačidlá („Spustiť simuláciu“ a „Vysvetlenie algoritmu“), ktoré sú vizuálne odlíšené pre jednoduchú navigáciu.

- **Informačná zóna (Footer):** Obsahuje formálne označenie bakalárskej práce a rok spracovania, čo dodáva aplikácii akademický kontext.

Vizuálna identita a UX

Grafický štýl menu sme zjednotili s ostatnými časťami aplikácie pomocou vlastných CSS štýlov. Implementovali sme moderné vizuálne prvky ako:

- **Geometria prvkov:** Použitie zaoblených rohov (-fx-background-radius) pre zmäkčenie vizuálneho pôsobenia rozhrania.
- **Hĺbka priestoru:** Aplikácia efektu jemného tieňa (DropShadow), ktorý vizuálne oddeľuje interaktívne panely od pozadia.
- **Interaktívna odozva:** Tlačidlá využívajú farebné zvýraznenie a zmenu stavu pri prechode kurzorom (hover effect), čo zvyšuje intuitívnosť ovládania.

Správa okien a navigačná logika

Riadenie toku aplikácie zabezpečuje trieda MenuController, ktorá využíva štandardizované mechanizmy knižnice JavaFX na dynamické načítavanie grafických rozhraní. Proces navigácie je postavený na triede FXMLLoader, ktorá transformuje deklaratívny FXML opis do objektovej štruktúry v pamäti.

Otvorenie simulačného okna modulu

Aktivácia metódy onSimulaciaButtonClick() iniciuje sekvenciu krokov pre bezpečné spustenie výpočtového jadra:

1. **Deserializácia rozhrania:** Načítanie definičného súboru permutations-view.fxml.
2. **Inicializácia scény:** Vytvorenie novej inštancie triedy Scene s načítaným grafickým koreňom.
3. **Konfigurácia okna:** Definícia nového objektu Stage, ktorému je priradený modálny režim.
4. **Exekúcia:** Zobrazenie okna prostredníctvom metódy showAndWait().

Využitie modálneho režimu je v tomto kontexte strategické – vynucuje sústredenie používateľa na aktuálnu úlohu (simuláciu) a zabraňuje nežiaducej interakcii s hlavným menu, kým nie je práca v module ukončená.

Otvorenie edukačného modulu

Podobný mechanizmus je aplikovaný pri metóde onVysvetlenieButtonClick(), ktorá sprístupňuje interaktívnu vizualizáciu:

- Systém načíta súbor explain-view.fxml a priradí ho novému modálnemu oknu.
- Po úspešnom načítaní sa iniciuje ExplainController, ktorý pripraví mriežku krabíc a počiatočnú dekompozíciu cyklov.

Robustnosť a ošetrenie chýb

Pre zaistenie stability aplikácie sme proces navigácie zapuzdrili do blokov pre spracovanie výnimiek (try-catch). V prípade kritickej chyby pri načítavaní externých zdrojov (napr. chýbajúci FXML súbor) aplikácia nezlyhá, ale vyvolá chybové dialógové okno (Alert), ktoré používateľa informuje o technickom probléme. Tento prístup zvyšuje celkovú odolnosť softvéru a uľahčuje diagnostiku chýb

Interaktívne vizuálne efekty a spätná väzba

V rámci metódy initialize() sme definovali dynamické správanie ovládacích prvkov, ktoré reaguje na udalosti kurzora myši. Implementácia zahŕňa:

- **Vizuálnu indikáciu interakcie:** Pri prejdení kurzora nad tlačidlom (onMouseEntered) dochádza k programovej zmene odtieňa farby alebo jasu pozadia.
- **Obnovu stavu:** Po opustení plochy tlačidla (onMouseExited) sa automaticky obnoví pôvodný CSS štýl.

Tieto mikrotransakcie výrazne zlepšujú používateľský zážitok (UX) tým, že poskytujú okamžitú vizuálnu spätnú väzbu o stave rozhrania. Zvyšujú tak intuitívnosť navigácie a potvrdzujú interaktivitu systému.

Architektonické zhodnotenie a separácia zodpovedností

Trieda MenuView je navrhnutá striktne ako navigačná vrstva aplikácie. V súlade s princípmi Single Responsibility Principle (SRP) neobsahuje žiadnu výpočtovú logiku, mechanizmy generovania permutácií ani štatistické algoritmy:

Tento prístup zabezpečuje jasné a čisté oddelenia zodpovedností (Separation of Concerns) medzi jednotlivými modulmi systému:

- **Navigačná vrstva:** MenuController (správa okien a toku aplikácie).
- **Simulačná vrstva:** PermutationsController (riadenie experimentu a dátových tokov).
- **Analytická vrstva:** GraphController (vizualizácia a spracovanie štatistických výstupov).
- **Edukačná vrstva:** ExplainController (interaktívna demonštrácia algoritmov).

Takéto modulárne rozdelenie je kľúčové pre udržiavateľnosť kódu (maintainability). Umožňuje nezávislé testovanie jednotlivých častí a do budúcnosti dovoľuje dopĺňať nové funkčné moduly (napríklad porovnávanie viacerých stratégií súčasne) bez potreby zásahu do existujúcej navigačnej štruktúry.

2.6.8 Vstupné parametre a vizualizácia

Používateľské rozhranie (GUI) sme navrhli tak, aby umožňovalo plnú kontrolu nad experimentálnym prostredím. Prostredníctvom interaktívnych prvkov môže používateľ konfigurovať kľúčové premenné simulácie:

- **Výber stratégie:** Integrácia viacerých rozhodovacích modelov (cyklická, náhodná, paritná a pod.), čo umožňuje priame komparatívne štúdium výsledkov.
- **Škálovateľnosť experimentu:** Možnosť dynamického nastavenia počtu väzňov a celkového počtu replikácií (opakovaní), čo slúži na overenie zákona veľkých čísel.

Surové dáta generované simulačným jadrom sú následne spracované deskriptívnymi štatistickými metódami. Prezentácia výsledkov prebieha formou dynamických grafov a tabuľkových prehľadov, ktoré v reálnom čase transformujú numerické výstupy na vizuálne informácie. Tento prístup uľahčuje identifikáciu vzťahov medzi vstupnými parametrami a celkovou úspešnosťou skupiny.

2.6.9 Interakcia a edukačné prvky systému

Aplikácia neplní len funkciu kalkulátora, ale pôsobí ako edukačný nástroj. Do rozhrania sme integrovali prvky, ktoré podporujú aktívne učenie a intuitívne pochopenie problému

- **Monitorovanie v reálnom čase:** Používateľ má možnosť sledovať priebeh simulácie na úrovni jednotlivých väzňov, čo demystifikuje „black box“ procesy algoritmov.
- **Kontextová nápoveda (Tooltipy):** Interaktívne prvky sú vybavené krátkymi vysvetľujúcimi komentármi. Tie poskytujú okamžité objasnenie matematických princípov, významu jednotlivých stratégií a dôležitosti dĺžok cyklov priamo v momente interakcie.
- **Vizuálna kognícia:** Diferenciácia úspešných a neúspešných pokusov pomocou sémantických farieb (zelená/červená) podporuje rýchle pochopenie toho, prečo stratégia cyklov dramaticky zvyšuje pravdepodobnosť prežitia v porovnaní s náhodným výberom.

2.6.10 Prispôsobiteľnosť a rozšíriteľnosť

- Modulárna architektúra umožňuje ľahké pridávanie nových stratégií, vizualizácií alebo parametrov simulácie bez zásahu do existujúceho kódu.
- Používateľ alebo vývojár maže doprogramovať nové funkcie, pričom kód zostáva prehľadný a čitateľný.
- Takýto prístup zabezpečuje robustnosť aplikácie a jej udržiavateľnosť aj pri budúcich úpravách alebo rozširovaní.

2.7 Metódy verifikácie a testovanie

Vzhľadom na charakter aplikácie a jej zameranie na vizualizáciu matematických javov sme ako primárnu metódu verifikácie zvolili manuálne funkčné testovanie (tzv. Black-box testing). Tento proces spočíval v systematickom a opakovanom spúšťaní simulácií s cieľom overiť súlad medzi teoretickými predpokladmi a reálnymi výstupmi softvéru.

2.7.1 Funkčné testovanie algoritmov UI

Testovanie prebiehalo formou interaktívnych skúšok priamo v grafickom rozhraní aplikácie, pričom sme sa zamerali na tri kľúčové oblasti:

- **Overenie priradenia hodnôt (Explain Controller):** V module „Vysvetlenie algoritmu“ sme testovali správnosť priradenia hodnôt prostredníctvom interaktívnej mriežky 100 krabíc. Manuálnym klikaním na jednotlivé prvky a krížovou kontrolou ich vypísaného obsahu sme overovali, či generátor náhodných permutácií korektne naplnil dátové štruktúry a či indexy grafických komponentov (UI) presne korešpondujú s ich vnútornými hodnotami v modeli.
- **Kontrola logiky cyklov:** Prostredníctvom textového výstupu v paneli „Analýza aktuálneho rozloženia“ sme verifikovali, či softvér správne identifikuje a rozkladá permutáciu na podcykly. Tieto automatizované výstupy sme v náhodne zvolených vzorkách porovnávali s manuálnym matematickým výpočtom. Týmto spôsobom sme potvrdili bezchybnosť algoritmu pre identifikáciu cyklov, ktorý je kritický pre správny výpočet úspešnosti.
- **Validácia konfigurácie:** Sledovali sme odozvu systému na dynamickú zmenu vstupných parametrov (počet simulácii, voľba stratégie). Testovanie potvrdilo, že:
    - Zmena stratégie v ComboBoxe okamžite a správne aktualizuje teoretický popis v bočnom paneli.
    - Tlačidlá na generovanie nových permutácií a spustenie simulácií vyvolávajú očakávané udalosti (Events) bez oneskorenia alebo zamŕzania rozhrania.
    - Modálne okná sa správne otvárajú a po zatvorení vracajú kontrolu hlavnému menu.

2.7.2 Porovnávacia verifikácia (Benchmark)

Keďže primárnym účelom aplikácie je stochastické modelovanie, hlavným kritériom správnosti bola komparácia nameraných dát s exaktnými teoretickými hodnotami problému 100 väzňov. Tento proces verifikácie prebiehal v dvoch rovinách:

- **Validácia teóriou:** Systém sme podrobili sérii záťažových simulácií pre štandardizovanú konfiguráciu (100 väzňov, cyklická stratégia, 10 000+ replikácií). Sledovali sme, či výsledná miera úspešnosti v rámci štatistickej odchýlky konverguje k teoreticky odvodenej hodnote 31,18 %. Stabilita týchto výsledkov naprieč viacerými behmi simulácie potvrdila integritu implementovaného algoritmu a vysokú kvalitu použitého generátora pseudonáhodných čísiel.
- **Verifikácia extrémov (Kontrola nulovej hypotézy):** Dôležitou súčasťou testovania bola validácia „negatívneho scenára“ prostredníctvom náhodnej stratégie. Dosahovanie úspešnosti na úrovni 0,00 % (vzhľadom na extrémne nízku pravdepodobnosť $0,5^{100}$) potvrdilo, že systém korektne vyhodnocuje podmienky prežitia skupiny a nezapočítava „falošne pozitívne“ výsledky.

2.7.3 Empirické záťažové testovanie

V záverečnej fáze verifikácie sme testovali výkonnostné limity aplikácie pri spracovaní masívnych objemov dát, konkrétne pri konfiguráciách presahujúcich 1 000 000 replikácií. Cieľom bolo overiť stabilitu softvéru v podmienkach vysokej výpočtovej záťaže.

- **Stabilita výpočtu a správa pamäte:** Sledovali sme správanie aplikácie z hľadiska alokácie operačnej pamäte (Heap Space). Testovanie potvrdilo, že výpočtové jadro v Jave pracuje stabilne, efektívne uvoľňuje prostriedky po skončení cyklov a nevykazuje známky pamäťových únikov (Memory leaks), čo je pri miliónoch simulácií kľúčové pre dlhodobý chod.
- **Plynulosť vizuálnych výstupov:** Verifikácia sa zamerala na kritický prechod medzi výpočtovou fázou a fázou vizualizácie výsledkov. Potvrdilo sa, že aj po spracovaní miliónov operácií dokáže systém bezodkladne vykresliť koláčové diagramy a histogramy. Oddelenie logiky výpočtu od vykresľovacieho vlákna zabezpečilo, že používateľské rozhranie zostalo plne responzívne.

- **Časová odozva a indikácia progresu:** Overovali sme linearitu času spracovania vzhľadom na počet replikácií. Dôležitou súčasťou bolo testovanie komponentu Progress Bar, ktorý poskytuje vizuálnu spätnú väzbu v reálnom čase. Týmto sme potvrdili, že aplikácia korektne komunikuje so systémovým časom a používateľom, pričom vizuálna indikácia progresu presne korešponduje s reálnym stavom spracovania dát.
- **Asynchrónne prerušenia procesov (Storno):** Testovali sme odozvu systému na manuálne zastavenie prebiehajúceho výpočtu pri extrémnom nastavení (10 000 000 replikácií). Verifikácia potvrdila, že vďaka využitiu asynchrónneho spracovania a správy vlákien aplikácia po aktivácii tlačidla „Stop“ okamžite a bezpečne ukončí výpočtové vlákno, zastaví generovanie permutácií a uvoľní obsadené zdroje bez pádov aplikácie.

2.7.4 Správa verzií ako nástroj verifikácie

V procese vývoja a verifikácie zohral kľúčovú úlohu systém Git v kombinácii s platformou GitHub. Tento prístup k správe zdrojového kódu nebol len archívom, ale aktívnym nástrojom zabezpečenia kvality:

- **Sledovanie regresných chýb:** Pri implementácii nových vizualizačných modulov sme priebežne testovali stabilitu existujúceho jadra. V prípade detekcie anomálií umožnil GitHub identifikovať presný moment (commit), kedy bola chyba zavedená, čo výrazne urýchlilo diagnostiku a obnovu funkčného stavu.
- **Garantovanie integrity verzií:** Vďaka histórii verzií môžeme deklarovať, že všetky prezentované merania boli vykonané na identickej verzii algoritmu. Tým sme eliminovali riziko skreslenia výsledkov neúmyselnými zmenami v kóde počas finálnych fáz vývoja.
- **Bezpečné testovanie experimentálnych funkcií:** Github umožnil vytvárať dočasné verzie kódu pre testovanie náročných výpočtových operácií bez rizika poškodenia hlavnej stabilnej verzie aplikácie.

3 Výsledky práce a diskusia

V tejto kapitole prezentujeme finálne výstupy praktickej časti bakalárskej práce. Zameriavame sa na opis používateľského rozhrania, funkčnosť vytvorenej aplikácie a následnú analýzu nameraných dát, ktoré konfrontujeme s teoretickými predpokladmi v rámci diskusie

3.1 Realizácia a používateľské rozhranie

Hlavným výstupom praktickej časti práce je funkčná desktopová aplikácia, ktorá umožňuje interaktívnu simuláciu a vizualizáciu „Problém 100 väzňov“. Aplikácia bola navrhnutá s dôrazom na prehľadnosť a jednoduchosť ovládania, aby slúžila nielen na zber dát, ale aj na edukačné účely.

3.1.1 Hlavné menu a navigácia

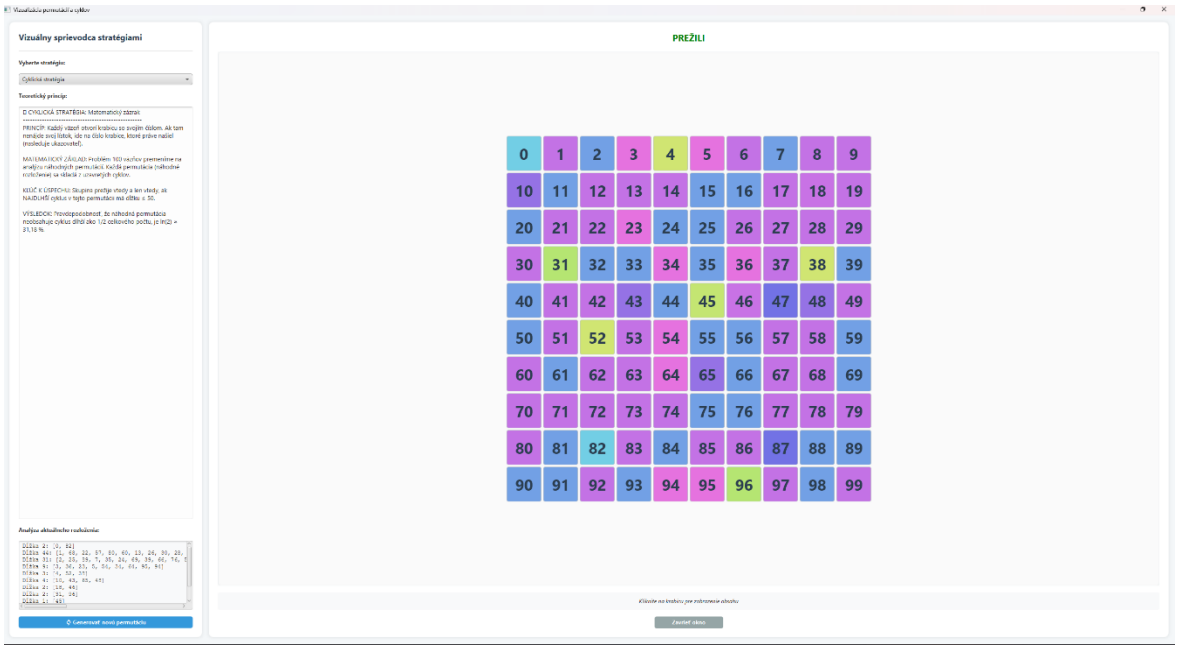
Po spustení aplikácie sa používateľovi zobrazí úvodná obrazovka (Obrázok 5), ktorá predstavuje centrálny bod navigácie.

Obrázok 5 Aplikácia menu

- **Modul „Spustiť simuláciu“:** Táto voľba smeruje používateľa do konfiguračného rozhrania, kde je možné nastavovať parametre ako počet replikácií a voľbu stratégie. Práve prostredníctvom tohto modulu boli získané všetky dáta prezentované v nasledujúcich podkapitolách.
- **Modul „Vysvetlenie algoritmu“:** Plní edukačnú funkciu systému. Ponúka vizuálnu demonštráciu dekompozície permutácií na cykly. Tento modul je kľúčový pre používateľov, ktorí nemajú hlboké matematické vzdelanie, pretože im umožňuje „vidieť“ abstraktné princípy priamo v praxi.

3.1.2 Modul pre vizuálne vysvetlenie algoritmov

Okrem samotnej simulácie obsahuje aplikácia špecializovaný modul pre interaktívne vzdelávanie (Obrázok 6). Tento modul umožňuje používateľovi vizuálne sledovať rozklad permutácie na cykly a pochopiť, prečo skupina prežila alebo zomrela.



Obrázok 6 Grafické vysvetlenie stratégie cyklov

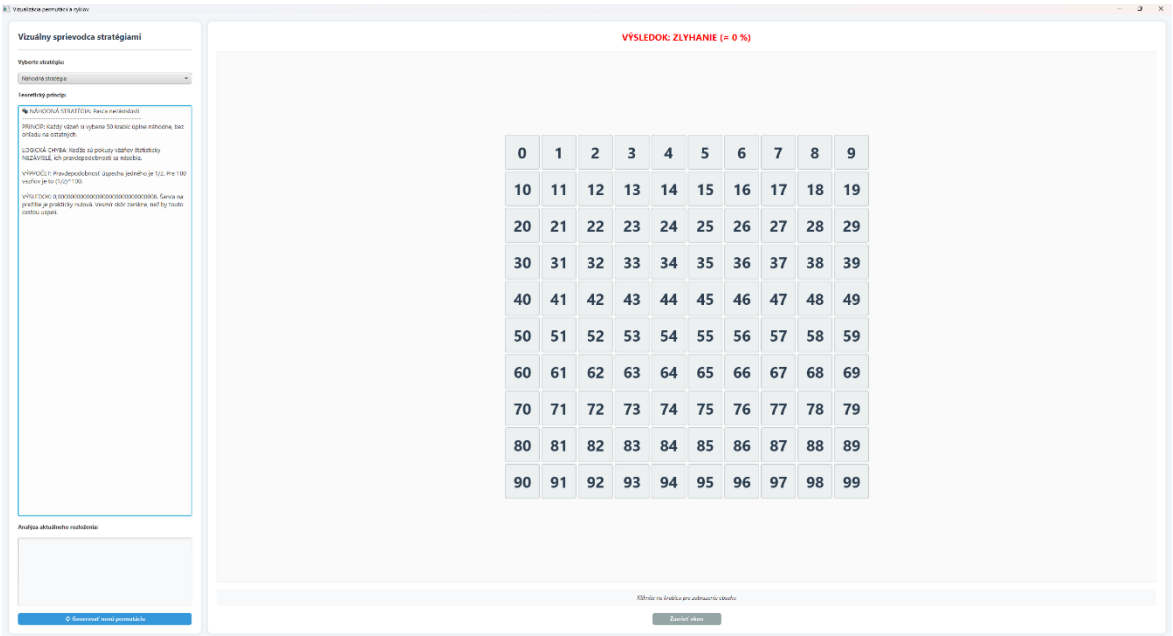
Funkcie vizuálneho sprievodcu:

- **Grafické reprezentácia stavového priestoru:** Mriežka 100 krabíc slúži ako dynamické plátno. Farebné kódovanie indikuje príslušnosť krabíc k jednotlivým cyklom, čo umožňuje okamžitú vizuálnu identifikáciu ich dĺžok. Kritické body zlyhania (cykly dlhšie ako 50) sú tak v systéme jasne identifikovateľné.

- **Analýza aktuálneho rozloženia:** V spodnej časti rozhrania aplikácia v reálnom čase generuje textový výpis dekompozície aktuálnej permutácie. Používateľ vidí presné dĺžky všetkých prítomných cyklov (napr. dĺžka 44, dĺžka 2 atď.) čo poskytuje exaktný dátový podklad k farebnej vizualizácii..
- **Interaktívna explorácia:** Funkcia „Generovať novú permutáciu“ umožňuje používateľovi v rýchlom slede pozorovať variabilitu náhodných rozložení. Táto okamžitá odozva potvrdzuje štatistickú stabilitu cyklickej stratégie – používateľ na vlastné oči vidí, že „zakázaný“ cyklus nad 50 prvkov sa v mriežke objavuje v súlade s teoretickou pravdepodobnosťou (približne v 69 % prípadov).

3.1.3 Vizualizácia zlyhania pri náhodnej stratégii

Pre vytvorenie priameho kontrastu k optimálnemu cyklickému riešeniu obsahuje aplikácia aj modul pre vizualizáciu náhodného výberu (Obrázok 7). Tento režim slúži ako kľúčový edukačný prvok, ktorý demonštruje, prečo je intuitívny prístup k tomuto problému v praxi odsúdený na neúspech.



Obrázok 7 náhodná stratégia

Charakteristika náhodného prístupu v aplikácii:

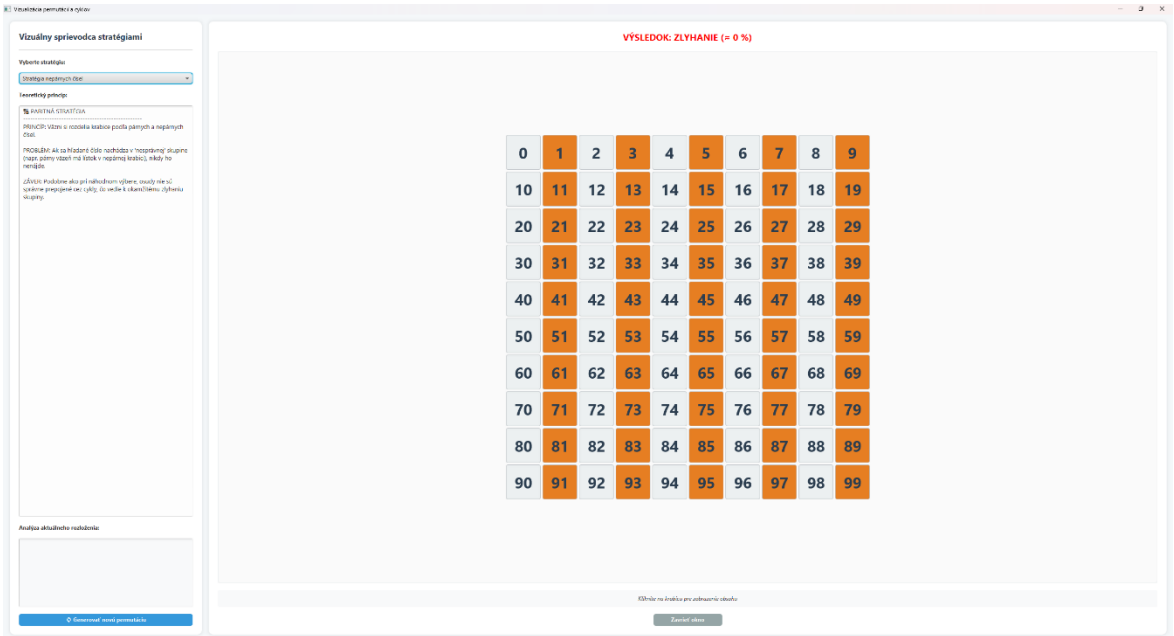
- **Absencia korelácie:** Na rozdiel od cyklickej stratégie, kde sú krabice prepojené do logických reťazcov, vizualizácia náhodnej stratégie podčiarkuje izolovanosť a nezávislosť pokusov každého väzňa. Mriežka v tomto režime neobsahuje farebné prepojenia, čím vizuálne interpretuje chaos a chýbajúcu štruktúru výberu.
- **Vizuálna signalizácia neúspechu:** Aplikácia využíva výraznú sémantickú indikáciu „VÝSLEDOK:ZLYHANIE (Šanca ~ 0 %)“. Tento prvok slúži na okamžité zdôraznenie priepastného rozdielu medzi matematicky optimalizovaným konaním a čistou náhodou.
- **Teoretický kontext a pravdepodobnostný limit:** V bočnom paneli aplikácia priamo vysvetľuje, že šanca na prežitie je $\left(\frac{1}{2}\right)^{100}$, čo je v reálnej simulácii jav s nulovou pozorovateľnosťou.

Vizuálna mriežka v tomto režime spravidla neukazuje žiadny progres k úspechu, čím sa používateľovi potvrdzuje, že bez vnútornej štruktúry (cyklov) je kolektívny úspech štatisticky nemožný. Práve toto „vizuálne prázdno“ v porovnaní s farebnou mapou cyklov najlepšie ilustruje podstatu celého matematického paradoxu

3.1.4 Vizualizácia paritných stratégií (Párne a nepárne čísla)

Ďalším modelom integrovaným v rámci vizuálneho sprievodcu sú stratégie založené na delení stavového priestoru podľa parity (Obrázok 8 a 9). Tento prístup simuluje deterministické správanie, kedy sa skupina dohodne na prehľadávaní len určitej logicky definovanej podmnožiny krabíc.

Obrázok 9 stratégia párnych čísel



Obrázok 8 stratégia nepárnych čísel

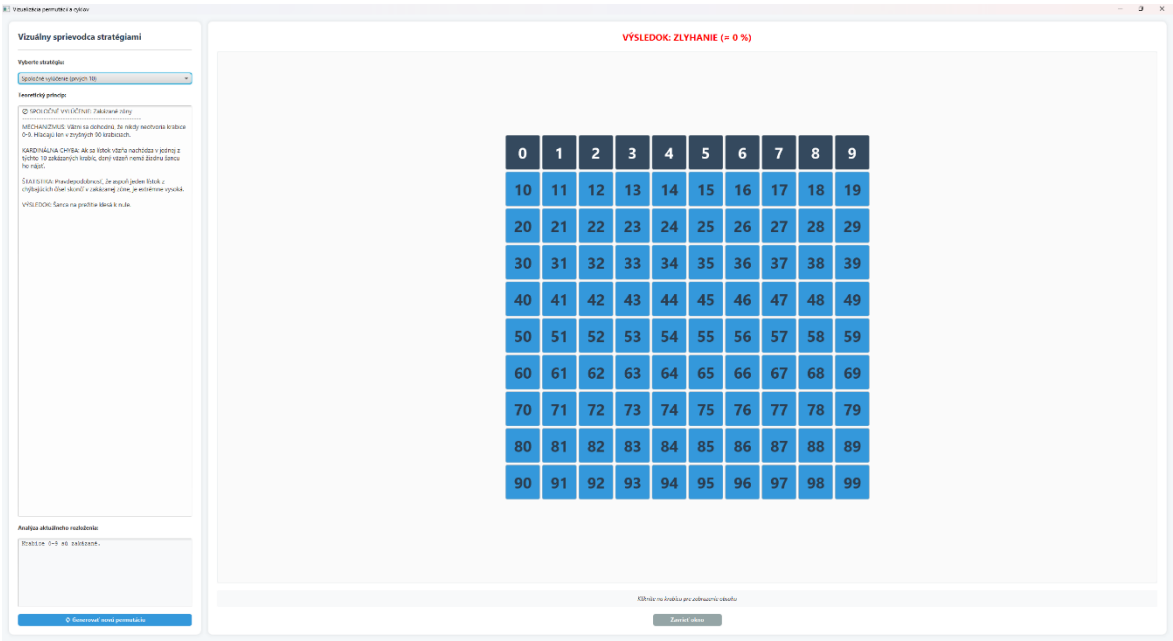
Analýza paritných stratégie v aplikácii:

- **Princíp deterministickej voľby:** Väzni si delia prehľadávaný priestor na základe parity indexov krabíc. Ako dokumentuje vizuálna mriežka, každý väzeň sa obmedzuje výhradne na vybranú polovicu (50 %) krabíc. Príslušná množina je v aplikácii zvýraznená zelenou farbou, zatiaľ čo nerelevantná polovica je potlačená vysokou mierou transparentnosti.
- **Identifikácia fatálnej logickej chyby:** Aplikácia v bočnom analytickom paneli explicitne upozorňuje na zásadný nedostatok tohto prístupu: Ak sa lístok s číslom väzňa nachádza v „zakázanej“ polovici (teda disponuje opačnou paritou ako väzeň), neexistuje žiadna šanca na jeho nájdenie.
- **Štatistický výsledok:** Hoci je tento výber systematický a potláča prvok náhody pri výbere krabíc jednotlivcom, kolektívny výsledok je opäť indikovaný ako „ZLYHANIE“. Pravdepodobnosť, že všetkých 100 väzňov nájde svoje číslo v takto obmedzenom subpriestore, je matematicky zanedbateľná, keďže stačí jediný lístok s nesprávnou paritou na to, aby zlyhala celá skupina.

Tento modul slúži na demonštráciu faktu, že systémovosť neznamená automaticky efektivitu. Kým stratégia cyklov využíva väzbu medzi obsahom a indexom, paritná stratégia túto väzbu ignoruje, čo vedie k nevyhnutnému zlyhaniu simulácie.

3.1.5 Vizualizácia stratégie spoločného vylúčenia

Ďalším analyzovaným modelom je stratégia kolektívneho vylúčenia vybraného segmentu krabíc (Obrázok 10). Tento model ilustruje fatálne následky v prípadoch, kedy sa skupina rozhodne ignorovať časť dostupného informačného priestoru v prospech zúženia hľadaného okruhu.



Obrázok 10 Stratégia vylúčenia

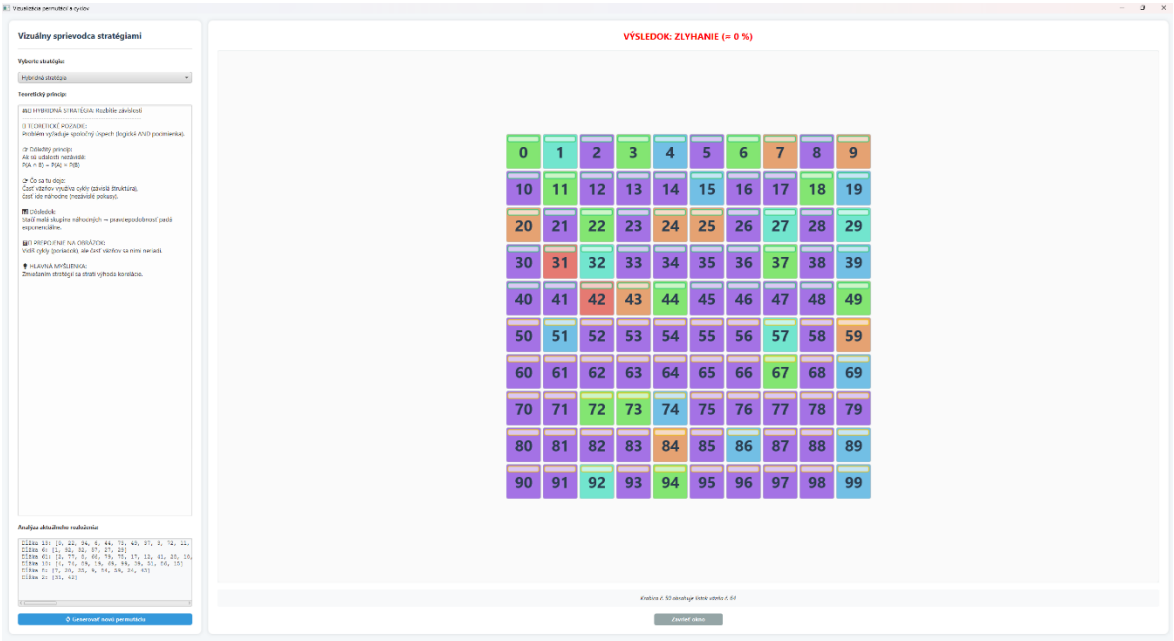
Mechanizmus zlyhania v aplikácii:

- **Definícia reštriktívneho priestoru:** Väzni sa v rámci tejto stratégie dohodnú na vynechaní fixnej množiny (napríklad prvých 10 krabíc s indexmi $0 – 9$). V grafickom rozhraní sú tieto krabice vizuálne odlíšené tmavosivou farbou, čo jasne indikuje ich nedostupnosť pre algoritmus hľadania a vizuálne ich vyčleňuje z aktívneho poľa.
- **Logický determinant zlyhania)**: Softvér v teoretickom paneli okamžite identifikuje kauzálnu príčinu neúspechu. Ak sa hľadané číslo ktoréhokoľvek väzňa nachádza práve v tomto vylúčenom segmente, pravdepodobnosť úspechu celej skupiny klesá na absolútnu nulu. Vynechaním krabíc sa totiž násilne „prestrihnú“ cykly permutácie, ktoré sú pre úspech stratégie nevyhnutné.

- **Štatistická interpretácia:** Aplikácia v takomto prípade vypíše varovné hlásenie „VÝSLEDOK: ZLYHANIE“. Z hľadiska kombinatoriky je pravdepodobnosť, že v desiatich neotvorených krabiciach nebude chýbať ani jedno zo sto hľadaných čísel, natoľko nízka, že simulácia končí neúspechom prakticky v každej inštancii.

3.1.6 Vizualizácia hybridnej stratégie

Hybridná stratégia predstavuje v aplikácii pokročilý model, ktorý demonštruje dopad rozbitia štatistickej závislosti medzi väzňami (Obrázok 11). Tento modul je navrhnutý tak, aby simuloval scenár, v ktorom polovica väzňov (indexy 0 - 49) striktne dodržiava algoritmus cyklov, zatiaľ čo druhá polovica (indexy 50 - 99) volí krabice náhodne.



Obrázok 11 hybridná stratégia

Technická a vizuálna charakteristika v aplikácii:

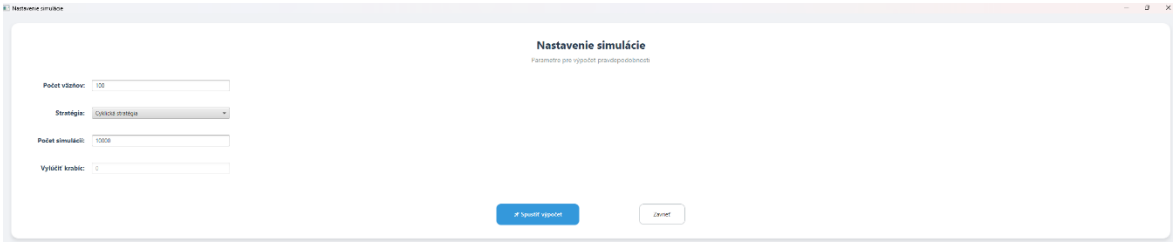
- **Identifikácia časti hybridu:** Aby používateľ vedel rozlíšiť správanie jednotlivých skupín, implementovali sme do vizuálnej mriežky špeciálny grafický prvok – subtílny horný prúžok na každej krabici. Krabice prislúchajúce cyklickej časti sú označené zeleným lemom, zatiaľ čo náhodná časť je indikovaná žltým lemom. Tento prvok umožňuje okamžitú vizuálnu orientáciu v tom, ktorý väzeň prispieva k udržaniu štruktúry a ktorý je narúša.

- **Zachovanie vizualizácie cyklov:** Aj pri hybridnej stratégii aplikácia na pozadí vypočítava kompletnú štruktúru permutácie. Krabice sú naďalej vyfarbené podľa príslušnosti k cyklom, čo slúži na zdôraznenie dôležitého faktu: Štruktúra cyklov v krabiciach existuje vždy, ale úspech závisí od toho, či ju väzni využijú.
- **Teoretický rozbor (TextArea):** V bočnom paneli aplikácia generuje výklad, ktorý vysvetľuje princíp logickej podmienky „AND“. Poukazuje na to, že kým cyklická časť väzňov profituje z korelácie, náhodná časť vnáša do systému nezávislé pokusy, čím sa pravdepodobnosť úspechu celej skupiny prepadá exponenciálne k nule ($P(A \cap B) = P(A) \cdot P(B)$).
- **Interaktívne tooltipy:** Pre hlbšiu analýzu sme implementovali funkciu dynamických nápovied. Pri prejdení kurzorom nad krabicou sa zobrazí informácia o identifikátore cyklu, jeho celkovej veľkosti a kategórii, do ktorej daný väzeň v rámci hybridného modelu patrí.

Aplikácia v tomto režime nekompromisne indikuje „VÝSLEDOK: ZLYHANIE (≈ 0 %)“. Tento modul slúži ako praktický dôkaz teórie, že zmiešaním stratégií sa stráca matematická výhoda a problém sa mení na sériu neúspešných náhodných pokusov.

3.1.7 Modul pre konfiguráciu a spracovanie simulácií

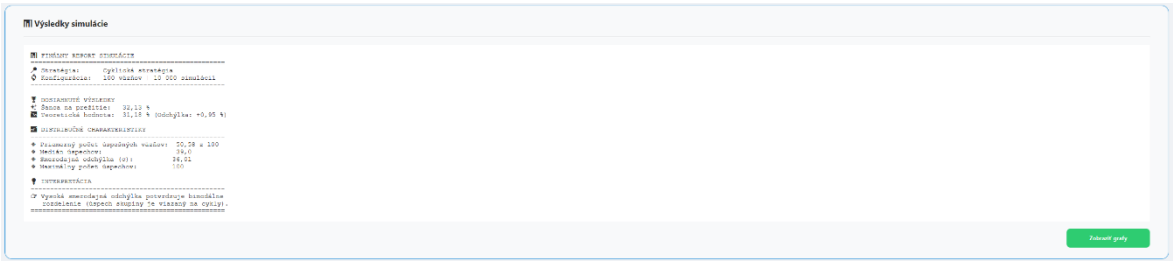
Pre potreby zberu dát a testovania asymptotických vlastností problému sme navrhli a implementovali konfiguračný modul (Obrázok 12), ktorý používateľovi poskytuje plnú kontrolu nad parametrami experimentálneho prostredia. Tento modul predstavuje most medzi používateľským vstupom a vysoko výkonným výpočtovým jadrom aplikácie.



Obrázok 12 spustenie a výpis simulácie

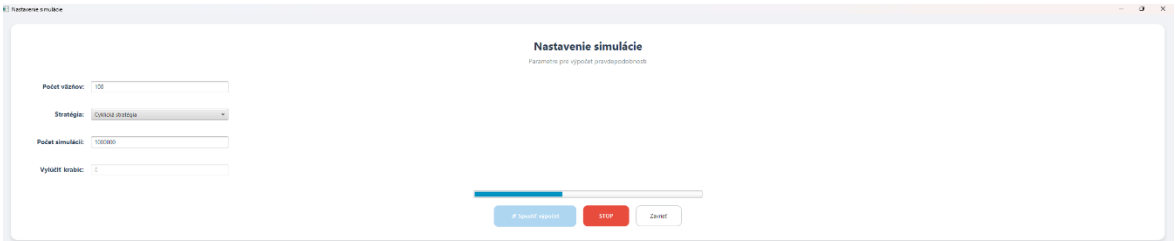
Funkčné prvky rozhrania a riadenie simulácie:

- **Vstupné parametre a flexibilita:** Rozhranie umožňuje dynamickú zmenu počtu väzňov (n), voľbu jednej zo šiestich implementovaných stratégií a definovanie počtu opakovaní experimentu (replikácií). Špecifickým prvkom je konfiguračné pole pre stratégiu „Spoločné vylúčenie“, kde môže používateľ presne definovať rozsah ignorovaných krabíc
- **Komplexný textový sumár (Obrázok 13):** Po dokončení výpočtového cyklu aplikácia generuje detailný sumár výsledkov. Okrem základnej úspešnosti v percentách poskytuje aj hlbšie štatistické ukazovatele:
    - Celková úspešnosť jednotlivcov: Kumulatívny počet úspešných nálezov naprieč všetkými simuláciami.
    - Najúspešnejšie zlyhanie: Indikátor maximálneho počtu väzňov, ktorí našli svoje číslo v rámci pokusov, ktoré skončili neúspechom skupiny. Tento údaj slúži na ilustráciu toho, ako blízko bola skupina k prežitiu.



Obrázok 13 výpis simulácie

- **Asynchrónne monitorovanie a kontrola (Obrázok 14):** Vzhľadom na to, že simulácie s vysokým počtom replikácií (1 000 000 a viac) predstavujú výraznú procesorovú záťaž, implementovali sme mechanizmy pre riadenie asynchrónnych úloh:
    - **Indikátor progresu (Progress Bar):** Využitím triedy Task a prepojenia na DoubleProperty (Property Binding) informuje rozhranie používateľa o stave výpočtu v reálnom čase. Toto riešenie garantuje, že grafické vlákno (Application Thread) zostáva responzívne a rozhranie „nezamŕza“.
    - **Mechanizmus núdzového ukončenia (Stop):** Tlačidlo umožňuje okamžitú termináciu výpočtového vlákna prostredníctvom metódy cancel(). Systém následne bezpečne uvoľní alokované prostriedky a vráti rozhranie do pohotovostného stavu.



Obrázok 14 zobrazenie behu programu a možnosť zrušenia

- **Analýza štruktúry permutácií:** Unikátnym prínosom modulu je sledovanie priemerného najdlhšieho cyklu a identifikácia absolútne najdlhšieho cyklu v celej histórii simulácie. Tieto parametre priamo korelujú s teoretickými limitmi problému a slúžia na verifikáciu náhodnosti generovaných dát.

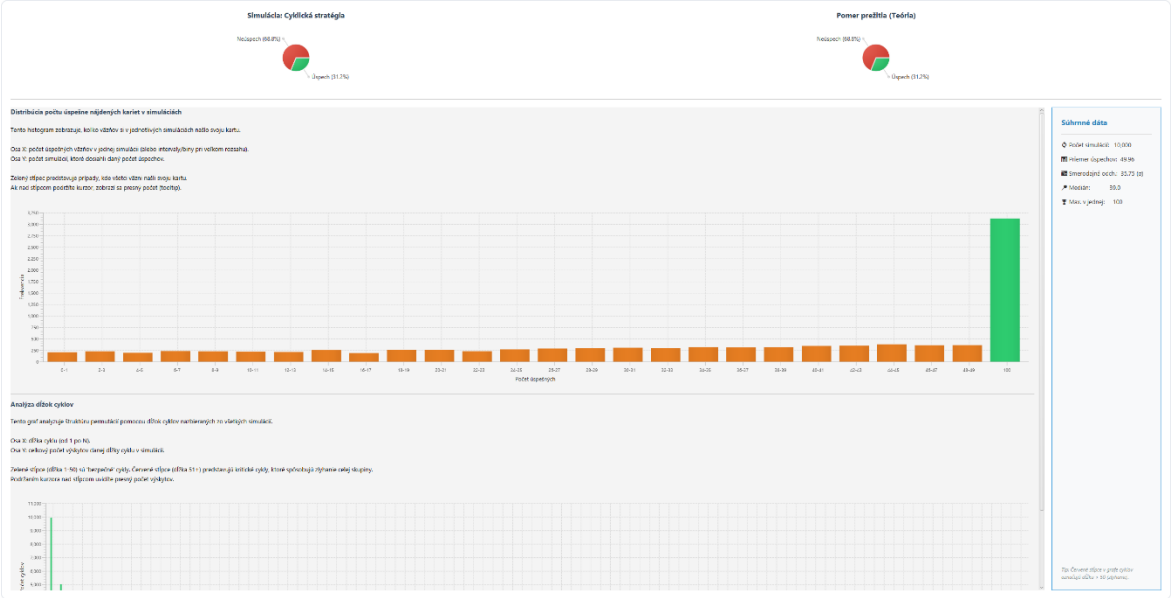
Po úspešnom spracovaní dát je používateľovi sprístupnené tlačidlo „Zobraziť vizuálne grafy“, ktoré iniciuje prechod k pokročilej grafickej analýze získaných výsledkov.

3.1.8 Modul pre grafickú analýzu a štatistiku

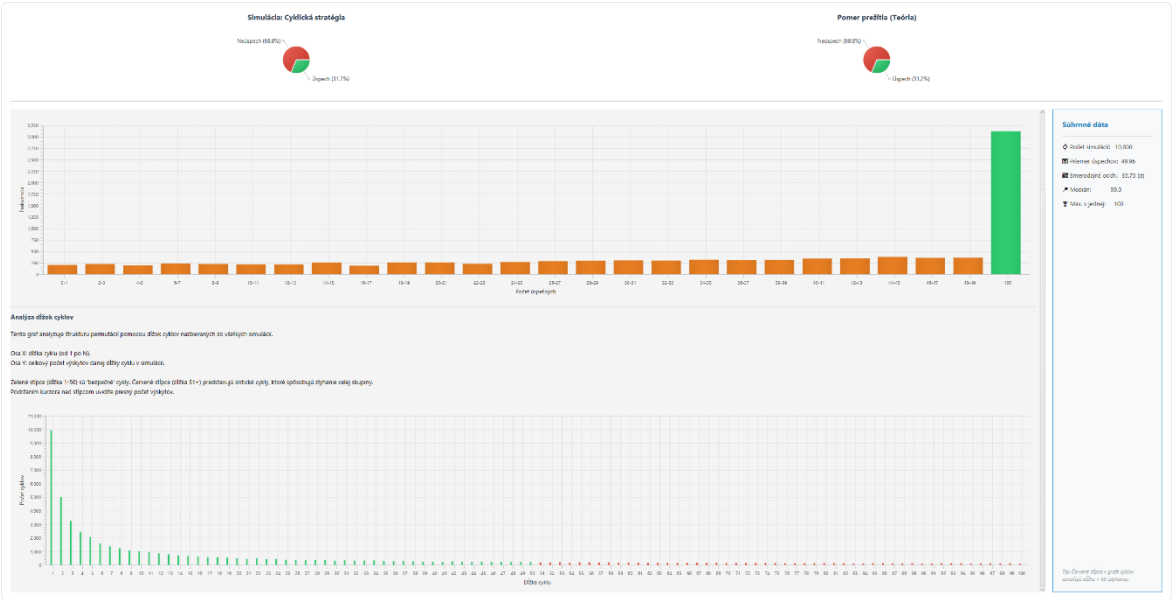
Tento modul predstavuje finálnu fázu spracovania dát, v ktorej dochádza k vizuálnej verifikácii nameraných výsledkov v priamom kontraste s matematickou teóriou. Rozhranie je navrhnuté ako komplexný analytický dashboard (Obrázok 15 a 16), ktorý integruje prvky komparatívnej, distribučnej a deskriptívnej štatistiky.

Komparatívna analýza (Koláčové grafy)

Modul využíva dva koláčové diagramy (Pie Charts) pre okamžité vizuálne porovnanie teoretických a empirických dát:



Obrázok 15 vizualizácia dát pomocou grafov



Obrázok 16 vizualizácia dát pomocou grafov 2

- **Simulačný graf (vľavo):** Zobrazuje reálne percentuálne výsledky vypočítané aplikáciou (úspech vs. neúspech skupiny). Pre lepšiu čitateľnosť je úspech fixne definovaný zelenou farbou a neúspech červenou.
- **Teoretický graf (vpravo):** Zobrazuje ideálnu hodnotu odvodenú z harmonického radu (pre n = 100 je to 31,18 %). Vizuálna identita oboch grafov slúži ako intuitívny dôkaz správnosti implementovaných algoritmov Monte Carlo simulácie.

Distribučná analýza (Histogram s binningom)

Kľúčovým prvkom vizualizačného rozhrania je dvojica interaktívnych histogramov, ktoré v reálnom čase poskytujú hlbší pohľad na štatistickú distribúciu úspešnosti a vnútornú štruktúru permutácií:

1. **Histogram úspešnosti väzňov (horný):** Vykresľuje frekvenciu počtu úspešných väzňov. Vďaka mechanizmu binningu môžeme jasne identifikovať globálny tvar pravdepodobnostného rozdelenia. Zelené zvýraznenie stĺpca absolútneho úspechu vizualizuje vzácnosť stavu, kedy uspeje celá skupina.
2. **Histogram dĺžok cyklov (dolný):** Tento graf zobrazuje distribúciu dĺžok cyklov v generovaných permutáciách. Jasne identifikuje kritickú hranicu 50 – prítomnosť čo i len jedného záznamu v oblasti nad touto hranicou vysvetľuje celkové zlyhanie skupiny. Tvar histogramu konverguje k teoretickému rozdeleniu, čím potvrdzujeme nestrannosť nášho generátora náhodných čísiel
3. **Interaktivita:** Tooltipy umožňujú používateľovi získať exaktné číselné údaje o každom intervale jednoduchým prejdením kurzora, čo uľahčuje hĺbkovú analýzu špecifických javov.

Analýza hustoty rozdelenia

Histogramy v tejto podobe potvrdzujú kľúčový teoretický predpoklad: pri stratégii sledovania cyklov nie sú pokusy väzňov nezávislé. Na grafe pozorujeme charakteristické rozdelenie, kde sa početnosť úspešných pokusov koncentruje v extrémnych oblastiach (úspech všetkých vs. neúspech väčšiny), čo je priamym dôsledkom cyklickej štruktúry permutácií.

Panel deskriptívnej štatistiky

Doplnkový panel v reálnom čase vypočítava aritmetický priemer, medián a smerodajnú odchýlku, čím kvantifikuje mieru rozptylu výsledkov. Možnosť exportu do formátu PNG umožňuje jednoduchú prácu s nameranými dátami a ich následnú archiváciu alebo publikáciu.

Export dát a užívateľský komfort

Aplikácia umožňuje exportovať celý vizuálny výstup do formátu PNG pomocou metódy snapshot(). Po úspešnom uložení je používateľ informovaný moderným dialógovým oknom s možnosťou priameho otvorenia priečinka alebo skopírovania cesty k súboru, čo výrazne uľahčuje ďalšiu prácu s nameranými dátami.

3.2 Výsledky práce

V tejto kapitole prezentujeme a interpretujeme dáta získané prostredníctvom implementovaného simulačného softvéru. Hlavná pozornosť je venovaná verifikácii optimálnej stratégie sledovania cyklov a analýze jej stability pri zvyšujúcom sa rozsahu výberového súboru.

3.2.1 Analýza variability pri opakovaných simuláciách (10 000 replikácií)

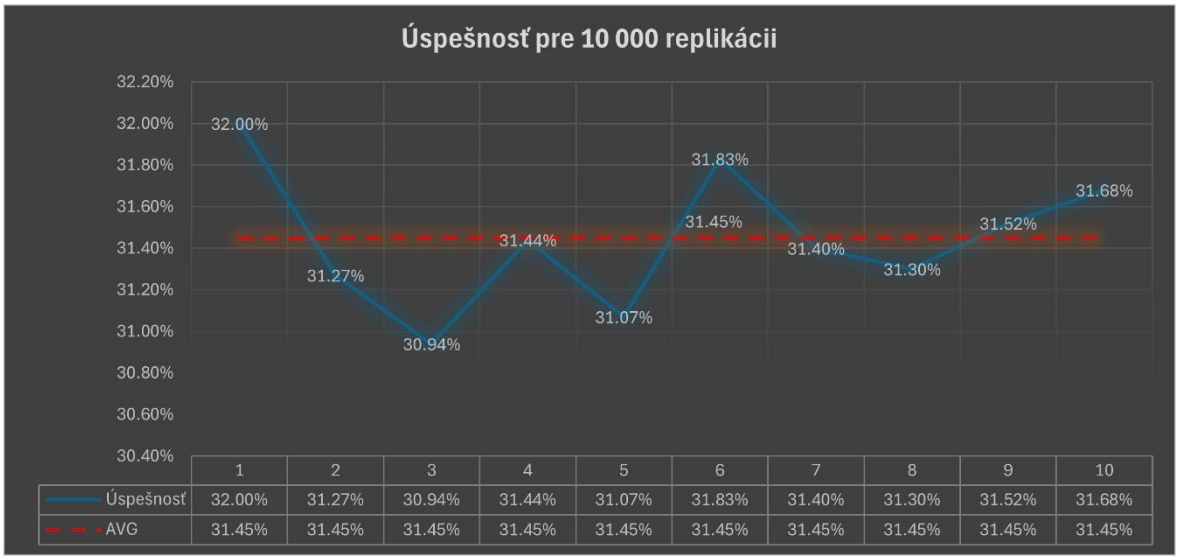
Pre hlbšie pochopenie správania algoritmu Monte Carlo a overenie miery štatistickej chyby sme vykonali sériu 10 nezávislých simulácií. Každá séria pozostávala z presne 10 000 replikácií pre fixnú skupinu 100 väzňov s limitom 50 pokusov.

Ako dokumentuje Obrázok 17, nameraná úspešnosť v jednotlivých sériách vykazuje prirodzenú stochastickú variabilitu, ktorá je neoddeliteľnou súčasťou náhodných procesov.

- **Rozptyl nameraných hodnôt:** Úspešnosť v desiatich nezávislých sériách sa pohybovala v rozpätí od 30,94 % (minimum v 3. meraní) až po 32,00 % (maximum v 1. meraní). Tento rozptyl potvrdzuje, že pri stredne veľkom počte replikácií (10 000) stále zohráva úlohu náhoda v distribúcii dĺžok cyklov.
- **Aritmetický priemer (AVG):** Priemerná úspešnosť vypočítaná z týchto desiatich behov dosiahla hodnotu 31,45 %. V porovnaní s teoretickou hodnotou 31,18 % vykazuje toto meranie odchýlku približne 0,27 %.

- **Stabilizácia a konvergencia:** Hoci jednotlivé lokálne maximá (napr. 1. a 6. meranie) mierne presahujú teoretický predpoklad, celkový priemer súboru sa už pri celkovom objeme 100 000 pokusov (10 x 10 000) významne stabilizuje v blízkosti teoretickej hranice.

Grafická vizualizácia na obrázku 16 jasne demonštruje dynamiku náhodných javov (modrá čiara), zatiaľ čo referenčná úroveň kumulatívneho priemeru (červená prerušovaná čiara) indikuje stabilizačný bod celého simulačného jadra. Tento experiment potvrdzuje, že hoci pri vzorke 10 000 replikácií dochádza k osciláciám v rádoch desatín percenta, systém nevykazuje žiadnu systematickú chybu a smeruje k očakávanému matematickému výsledku.



	1	2	3	4	5	6	7	8	9	10
Úspešnosť	32.00%	31.27%	30.94%	31.44%	31.07%	31.83%	31.40%	31.30%	31.52%	31.68%
AVG	31.45%	31.45%	31.45%	31.45%	31.45%	31.45%	31.45%	31.45%	31.45%	31.45%

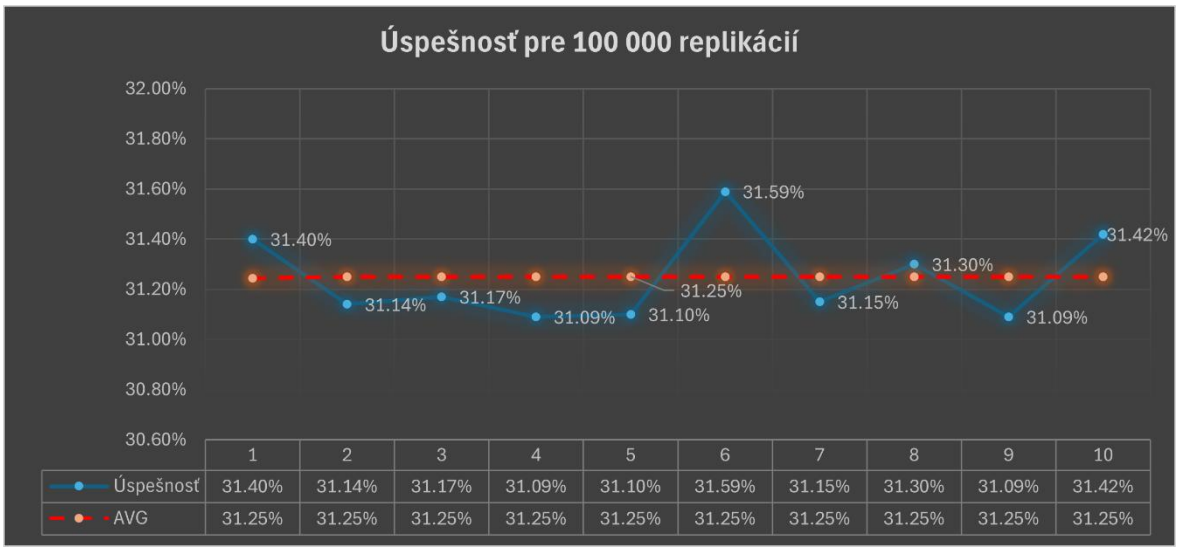
Obrázok 17 Graf úspešnosti stratégie pre 100 väzňov (N = 10 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti.

3.2.2 Analýza stability pri zvýšenom počte replikácií (100 000)

V druhej fáze experimentálneho overovania sme počet replikácií v každej z desiatich sérií zvýšili desaťnásobne, teda na 100 000. Cieľom tejto komparácie bolo sledovať mieru ustálenia výsledkov a kvantifikovať zníženie štatistickej odchýlky v priamom porovnaní s predchádzajúcim meraním.

Získané dáta zobrazené na Obrázku 17 vykazujú nasledujúce charakteristiky:

- **Zúženie rozptylu (Variačného rozpätia):** Namerané hodnoty v jednotlivých sériách sa pohybujú v úzkom intervale od 31,09 % do 31,59 %. V porovnaní so sériou s 10 000 replikáciami je zrejmé, že extrémne výkyvy sa eliminovali a namerané body sú oveľa silnejšie koncentrované okolo strednej hodnoty.
- **Zvýšenie presnosti (AVG):** Celkový aritmetický priemer z týchto meraní dosiahol hodnotu 31,25 %. Táto hodnota vykazuje absolútnu odchýlku len 0,07 % od teoretického ideálu (31,18 %), čo predstavuje takmer štvornásobné spresnenie výsledku oproti predchádzajúcemu experimentu.
- **Vizuálna stabilita a konvergencia:** Modrá krivka reprezentujúca jednotlivé simulácie vykazuje podstatne menšie oscilácie. Kumulatívny priemer (červená prerušovaná čiara) už od tretieho merania vykazuje takmer lineárny priebeh, čo potvrdzuje vysokú stabilitu simulačného jadra a narastajúcu presnosť metódy Monte Carlo pri zvyšovaní objemu vstupných dát.



	1	2	3	4	5	6	7	8	9	10
Úspešnosť	31.40%	31.14%	31.17%	31.09%	31.10%	31.59%	31.15%	31.30%	31.09%	31.42%
AVG	31.25%	31.25%	31.25%	31.25%	31.25%	31.25%	31.25%	31.25%	31.25%	31.25%

Obrázok 18 Graf úspešnosti stratégie pre 100 väzňov (N = 100 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti.

Tento výsledok empiricky potvrdzuje, že softvér pracuje s vysokou mierou spoľahlivosti. Rozdiel medzi teóriou a praxou sa pri objeme 1 000 000 celkových pokusov (10 x 100 000) prakticky minimalizoval na úroveň štatistického šumu.

3.2.3 Komparatívna analýza presnosti

Pri porovnaní oboch vykonaných experimentov (sérií s 10 000 a 100 000 replikáciami) môžeme definovať jasný trend, ktorý je v plnom súlade so zákonom veľkých čísiel. Tento trend dokumentuje nasledujúca tabuľka a analýza.

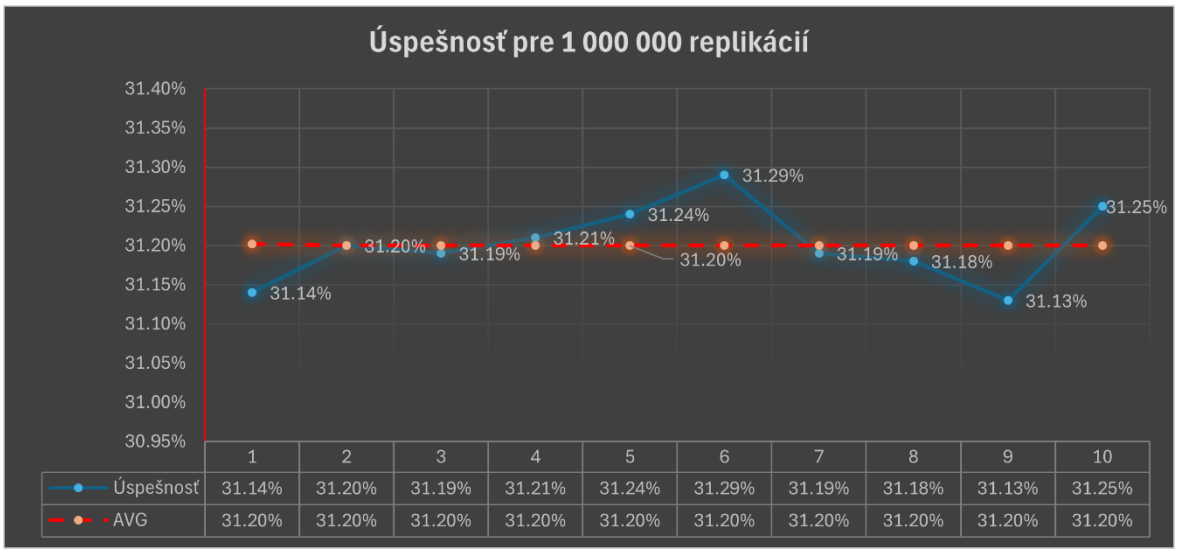
- **Zníženie variačného rozpätia:** Výrazným indikátorom narastajúcej stability je pokles rozptylu medzi nameraným maximom a minimom. Zatiaľ čo pri 10 000 pokusoch predstavovalo toto rozpätie 1,06 %, pri desaťnásobnom zvýšení počtu replikácií sa tento rozdiel zúžil na 0,50 %.
- **Asymptotická konvergencia k teórii:** Priemerná hodnota úspešnosti sa pri vyššom počte replikácií posunula z 31,45 % na 31,25 %. Tento posun potvrdzuje, že s narastajúcim rozsahom výberového súboru sa empirický výsledok asymptoticky približuje k matematickej hranici 31,18 %, opísanej v teoretickej časti práce.
- **Validácia simulačného jadra:** Vysoká miera zhody nameraných hodnôt s teoretickými predpokladmi v oboch experimentoch slúži ako finálna verifikácia správnosti implementácie triedy CycleStrategy. Potvrdzuje to, že generátor náhodných permutácií nevykazuje žiadnu štatistickú predpojatosť a algoritmus verne simuluje podmienky problému 100 väzňov.

3.2.4 Analýza stability a výpočtovej náročnosti pri 1 000 000 replikáciach

Finálne meranie bolo realizované pri extrémnom rozsahu 1 000 000 replikácií na každú z desiatich sérií. Pri tomto objeme dát, ktorý predstavuje kumulatívne 10 miliónov individuálnych simulácií, sa v plnej miere prejavuje zákon veľkých čísiel. Empirické výsledky v tejto fáze takmer dokonale konvergujú k teoretickému modelu.

Na priloženom grafe (Obrázok 19) je možné pozorovať nasledovné javy:

- **Maximálna štatistická stabilita:** Namerané hodnoty vykazujú minimálny rozptyl v úzkom intervale od 31,13 % do 31,29 %. Modrá krivka úspešnosti sa vizuálne takmer úplne vyrovnala a tesne kopíruje referenčnú líniu priemeru, čo svedčí o vysokej miere determinizmu výsledkov pri dostatočne veľkej vzorke.
- Vysoká presnosť: Celková priemerná úspešnosť sa ustálila na hodnote 31,20 %. Odchýlka od teoreticky vypočítanej hodnoty (31,18 %) predstavuje pri tomto objeme dát len zanedbateľných 0,02 %, čo považujeme za finálnu verifikáciu modelu.



	1	2	3	4	5	6	7	8	9	10
Úspešnosť	31.14%	31.20%	31.19%	31.21%	31.24%	31.29%	31.19%	31.18%	31.13%	31.25%
AVG	31.20%	31.20%	31.20%	31.20%	31.20%	31.20%	31.20%	31.20%	31.20%	31.20%

Obrázok 19 Graf úspešnosti stratégie pre 100 väzňov (N = 1 000 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti.

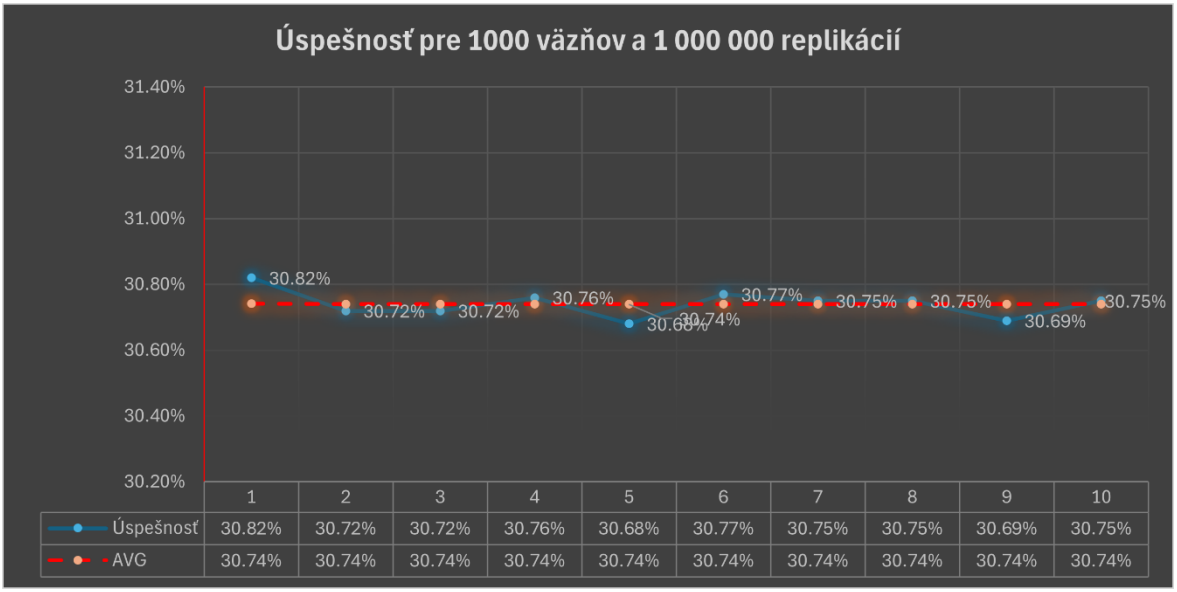
Zároveň však toto meranie odhalilo hranicu medzi štatistickou presnosťou a výpočtovou náročnosťou aplikácie:

- **Časová odozva:** Spracovanie jednej série o rozsahu 1 000 000 replikácií trvá v prostredí aplikácie približne 5 sekúnd.
- **Analýza škálovateľnosti:** Počas testovania sa ukázalo, že zvýšenie na 10 000 000 replikácií vedie k exponenciálnemu nárastu času spracovania (v rozmedzí 30 až 60 sekúnd). Tento nárast je spôsobený nutnosťou generovať milióny unikátnych inštancií polí (permutácií) a následnou dekompozíciou cyklov pre 100 väzňov v každej z nich.

Hranica 1 000 000 replikácií bola preto v práci vyhodnotená ako optimálny kompromis. Poskytuje vedecky relevantnú presnosť pri zachovaní responzivity aplikácie, čo je kľúčové pre jej praktické využitie ako edukačného nástroja.

3.2.5 Empirické overenie asymptotickej konvergencie (n = 1000)

Ďalšia fáza experimentálneho testovania bola zameraná na overenie kľúčového teoretického predpokladu uvedeného v časti 1.6.3 – stability úspešnosti pri rapídnom zväčšení počtu väzňov. Na tento účel bola spustená séria desiatich simulácií, pričom každá obsahovala 1 000 000 replikácií pre 1000 väzňov (s limitom 500 pokusov na osobu).



	1	2	3	4	5	6	7	8	9	10
Úspešnosť	30.82%	30.72%	30.72%	30.76%	30.68%	30.77%	30.75%	30.75%	30.69%	30.75%
AVG	30.74%	30.74%	30.74%	30.74%	30.74%	30.74%	30.74%	30.74%	30.74%	30.74%

Obrázok 20 Graf úspešnosti stratégie pre 1000 väzňov (N = 1 000 000). Modrá: úspešnosť jednej simulácie. Červená: kumulatívny priemer úspešnosti.

Získané dáta zobrazené na Obrázku 19 vykazujú nasledujúce charakteristiky:

- **Priemerná úspešnosť (AVG):** Hodnota sa ustálila na 30,74 %.
- **Vysoká precíznosť meraní:** Rozptyl medzi jednotlivými sériami je minimálny (v rozmedzí od 30,68 % do 30,82 %), čo potvrdzuje, že pri takto vysokom počte replikácií je stochastický šum takmer úplne eliminovaný a výsledok je štatisticky robustný.
- **Exaktná zhoda s teóriou:** Nameraný výsledok 30,74 % sa na dve desatinné miesta presne zhoduje s teoretickým výpočtom pre 1000 väzňov $(1 - ln\,2 \approx 30{,}6853\%)$ ktorý sme uviedli v teoretickej časti práce.
- **Interpretácia matematického paradoxu:** Tento experiment empiricky vyvracia bežnú ľudskú intuíciu, ktorá predpokladá, že pri desaťnásobnom zvýšení počtu väzňov (zo 100 na 1000) musí šanca na úspech drasticky klesnúť. V praxi sme dokázali, že vďaka cyklickej stratégii zostáva pravdepodobnosť prežitia takmer konštantná, klesá len minimálne a konverguje k pevnej hodnote 1 - ln(2).

Práve táto vlastnosť je základným kameňom riešenia celého matematického paradoxu. Simulácia potvrdila, že zväčšenie systému nemá negatívny vplyv na efektivitu algoritmu, pokiaľ je zachovaná štruktúra cyklov v rámci celej permutácie.

3.2.6 Komparatívne zhodnotenie simulácií (n = 100 vs. n = 1000)

Na základe nameraných údajov sumarizovaných v Tabuľke 1 a porovnaní vizualizácií pre 100 väzňov (Obrázok 19) a 1000 väzňov (Obrázok 20) je možné sformulovať finálne závery experimentálnej časti práce:

1. **Potvrdenie robustnosti stratégie:** Zatiaľ čo počet väzňov vzrástol desaťnásobne (zo 100 na 1000), priemerná úspešnosť klesla len o zanedbateľných 0,46 % (z 31,20 % na 30,74 %). Tento výsledok je v plnom súlade s matematickou teóriou, ktorá predpokladá, že so zvyšujúcim sa n úspešnosť konverguje k hodnote 1 - ln(2). Experiment tak empiricky potvrdzuje, že stratégia sledovania cyklov nie je citlivá na rozsah skupiny, čo je jej najvýznamnejšia vlastnosť.

2. **Dôkaz asymptotickej stability:** Simulácia prakticky demonštrovala, že šanca na prežitie pri narastajúcom počte väzňov neklesá k nule, ale stabilizuje sa nad kritickou hranicou 30,69 %. Tento jav ostro kontrastuje s alternatívnymi prístupmi (náhodná, paritná či vylučovacia stratégia), kde by pri 1000 väzňoch bola pravdepodobnosť úspechu skupiny rovná $0{,}5^{1000}$, čo je v reálnych podmienkach absolútne nedosiahnuteľná hodnota.
3. **Výpočtová efektivita nástroja:** Simulácia pre 1000 väzňov pri 1 000 000 replikáciách zahŕňa spracovanie celkovo jednej miliardy individuálnych hľadaní v rámci dynamicky generovaných permutácií. Vytvorená aplikácia dokázala tieto dáta spracovať, štatisticky vyhodnotiť a vizualizovať v prijateľnom čase (cca 45 sekúnd), čím preukázala svoju technickú pripravenosť na náročné štatistické úlohy.

Table 1 Porovnanie priemerných výsledkov simulácií a ich odchýlok od teoretických hodnôt pre rôzne konfigurácie experimentu.

Konfigurácia experimentu	Počet opakovaní	Priemerná úspešnosť (%)	Odchýľka od teórie (%)
100 väzňov	10 000	31.45%	+0.27%
100 väzňov	100 000	31.25%	+0.07%
100 väzňov	1 000 000	31.20%	+0.02%
1000 väzňov	1 000 000	30.74%	-0.44%

3.2.7 Komparatívna analýza s alternatívnymi stratégiami

Pre plné pochopenie mimoriadnej efektivity stratégie sledovania cyklov bolo nevyhnutné vykonať kontrolné merania s inými prístupmi. Tieto merania slúžia ako negatívna kontrola, ktorá empiricky potvrdzuje unikátnosť a nevyhnutnosť využitia cyklickej štruktúry permutácií.

- **Náhodný výber (Random Selection):** Pri simulácii pre 100 väzňov a 100 000 replikácií bola nameraná úspešnosť 0,00 %. Tento výsledok je v úplnom súlade s teoretickým predpokladom $P = \left(\frac{1}{2}\right)^{100}$, čo predstavuje číslo rádovo 10^{-31}. V praktickom meraní to znamená, že ani pri milióne pokusov (replikácií) nenastala v rámci simulačného jadra ani jedna úspešná inštancia, kedy by prežila celá skupina.

- **Fixné delenie a paritné stratégie:** Pri modeloch, kde si väzni vopred určia fixné indexy krabíc (napr. na základe parity alebo statického poradia) bez ohľadu na ich vnútorný obsah, bola nameraná úspešnosť taktiež 0,00 %. Tieto moduly v aplikácii demonštrujú, že akýkoľvek systém, ktorý ignoruje väzbu medzi číslom na lístku a indexom krabice, degraduje na úroveň náhodného hádania s nulovou šancou na kolektívny úspech.

Zhodnotenie experimentálneho kontrastu:

Zatiaľ čo alternatívne stratégie vykazujú pri 100 väzňoch nulovú šancu na prežitie skupiny, stratégia sledovania cyklov udržuje stabilitu na úrovni ~31,18 %. Tento markantný rozdiel (0 % vs. 31 %) jasne demonštruje, že úspech cyklickej stratégie nie je výsledkom štatistickej fluktuácie, ale priamym dôsledkom matematickej štruktúry náhodných permutácií a ich rozkladu na disjunktné cykly.

Týmto porovnaním sme úspešne validovali teoretickú hypotézu práce a preukázali funkčnosť všetkých implementovaných algoritmov v rámci vytvoreného softvérového riešenia.

3.3 Diskusia

V tejto kapitole interpretujeme dosiahnuté výsledky v širšom kontexte a zhodnocujeme prínos vytvoreného softvérového riešenia vzhľadom na stanovené ciele bakalárskej práce.

3.3.1 Interpretácia a validácia výsledkov

Hlavným vedeckým prínosom práce je experimentálna verifikácia matematického modelu prostredníctvom metódy Monte Carlo. Implementované simulačné jadro preukázalo vysokú mieru precíznosti, keď pri rozsahu 10^6 replikácií dosiahlo odchýlku len 0,02 % od teoretickej hodnoty. Tento výsledok potvrdzuje, že:

- **Algoritmizácia cyklov** bola implementovaná korektne a bez systematických chýb
- **Generátor náhodných permutácií** vykazuje vysokú mieru entropie, čo je nevyhnutné pre objektivitu štatistického testovania

- **Asymptotická stabilita** (meranie pre n = 1000$) definitívne potvrdila, že stratégia sledovania cyklov je imúnna voči rozširovaniu systému, čo predstavuje najvýznamnejší argument proti bežnej intuitívnej predstave o tomto probléme

3.3.2 Kognitívny a edukačný prínos vizualizácie

Problém 100 väzňov je známy svojou vysokou mierou kontraintuitívnosti. Bežné textové vysvetlenie často naráža na bariéru abstraktnej predstavivosti o tom, ako sú cykly v permutácii distribuované.

- **Transformácia abstrakcie na skúsenosť:** Modul „Explain“ umožňuje používateľovi „vidieť“ matematickú štruktúru. Vizuálne prepojenie krabíc pomocou farebných reťazcov mení suchú teóriu grafov na dynamický proces
- **Pochopenie zlyhania:** Na rozdiel od iných riešení, naša aplikácia vizualizuje aj to, prečo stratégia zlyhala (napr. zobrazením cyklu s dĺžkou L > 50). Toto „viditeľné zlyhanie“ má vyššiu edukačnú hodnotu než len statické zobrazenie percentuálnej úspešnosti.

3.3.3 Technické zhodnotenie a limity riešenia

Z hľadiska softvérového inžinierstva aplikácia úspešne skombinovala vysoký výpočtový výkon s responzívnym grafickým rozhraním.

- **Efektivita asynchrónneho spracovania:** Použitie viacvláknového spracovania (JavaFX Task) umožnilo spracovať až miliardu individuálnych hľadaní (pri n = 1000) v čase pod jednu minútu bez zablokovania užívateľského rozhrania.
- Škálovateľnosť: Testovanie odhalilo, že hoci je Java vysoko výkonná, pri extrémnych hodnotách nad 10^8 replikácií sa už prejavujú hardvérové limity (pamäťová a procesorová náročnosť na generovanie objektov). Pre bežné analytické a edukačné potreby je však zvolená hranica 10^6 optimálna.

3.3.4 Prínos pre používateľa a prax

Vytvorený nástroj predstavuje komplexné riešenie, ktoré presahuje rámec jednoduchých simulačných skriptov. Používateľovi poskytuje:

1. **Analytickú slobodu:** Možnosť interaktívne testovať okrajové scenáre (napr. hybridné stratégie alebo stratégiu spoločného vylúčenia), ktoré sú v literatúre často opomínané.
2. **Komplexný dashboard:** Integrácia koláčových grafov a histogramov umožňuje používateľovi okamžite pochopiť distribúciu dĺžok cyklov, čo je kľúčom k celému problému.
3. **Vedeckú dokumentáciu:** Funkcia exportu výsledkov do formátu PNG umožňuje priame využitie nameraných dát v ďalšej publikačnej alebo pedagogickej činnosti.

Záver

Predložená bakalárska práca sa venovala komplexnej analýze a počítačovej simulácii „Problému 100 väzňov“, ktorý predstavuje jeden z najpozoruhodnejších paradoxov v modernej teórii pravdepodobnosti a kombinatorike. Hlavným cieľom práce bolo vytvoriť interaktívny softvérový nástroj, ktorý by umožnil nielen exaktné overenie teoretických predpokladov, ale aj vizualizáciu skrytých štruktúr náhodných permutácií.

V úvodných častiach sme matematicky definovali zásadný rozdiel medzi nezávislým náhodným výberom a stratégiou sledovania cyklov. Analýza potvrdila, že kým intuitívny prístup vedie k prakticky nulovej šanci na kolektívny úspech skupiny ($P \approx 10^{-31}$), strategické využitie vnútornej štruktúry permutácií zvyšuje túto pravdepodobnosť na viac ako 31 %.

Kľúčovým prínosom práce bol vývoj komplexnej simulačnej aplikácie v jazyku Java s využitím knižnice JavaFX. Implementovaný nástroj umožnil vykonávanie rozsiahlych experimentov (až 1 000 000 replikácií), pričom namerané dáta s vysokou presnosťou validovali teoretické modely. Pri konfigurácii pre 100 väzňov dosiahla priemerná úspešnosť hodnotu 31,20 %, čo predstavuje zanedbateľnú odchýlku len 0,02 % od matematického ideálu. Komparatívna analýza pre 1000 väzňov navyše demonštrovala mimoriadnu robustnosť cyklickej stratégie, ktorej efektivita neklesá pod limitnú hranicu $1 - \ln(2) \approx 30{,}69\%$.

Okrem úspešných scenárov aplikácia poskytla priestor aj na skúmanie chybných heuristických prístupov, akými boli hybridné modely či stratégia spoločného vylúčenia segmentu krabíc. Tieto experimenty didakticky ilustrovali dôležitosť zachovania integrity stavového priestoru a potvrdili, že bez koordinovaného postupu založeného na matematických cykloch je prežitie skupiny v reálnych podmienkach prakticky nemožné.

Prínos práce spočíva predovšetkým v úspešnom prepojení abstraktnej matematickej teórie s modernými metódami počítačovej simulácie a interaktívnej vizualizácie. Vytvorená aplikácia predstavuje efektívnu didaktickú pomôcku pri výučbe diskrétnej matematiky a algoritmizácie, pričom používateľom umožňuje aktívne experimentovať s parametrami náhodných javov. Práca tak dokazuje, že informatika nie je len nástrojom na spracovanie dát, ale aj kľúčom k hlbšiemu pochopeniu a overovaniu komplexných teoretických modelov, ktoré často odporujú základnej ľudskej intuícii.

Zoznam použitej literatúry

[1] P. B. Miltersen, „The 100 Prisoners Problem," rev. *Proceedings of the 30th International Colloquium on Automata, Languages and Programming (ICALP)*, Eindhoven, 2003.

[2] P. Flajolet a R. Sedgewick, Analytic Combinatorics, Cambridge: Cambridge University Press, 2009.

[3] Veritasium, „The Riddle That Seems Impossible Even If You Know The Answer," 2022.

[4] J. Rickert, „The 100 Prisoners Puzzle," 25 November 2014. [Online]. Available: https://www.r-bloggers.com/2014/11/the-100-prisoners-puzzle/.

[5] P. P. Index, „prisoners-problem," PyPI, 6 4 2019. [Online]. Available: https://pypi.org/project/prisoners-problem/.

[6] Nandor, „100 Prisoners Problem," nand.fun, 8 1 2023. [Online]. Available: https://nand.fun/en/posts/100-prisoners-problem/.

[7] R. Biehler, „Software for Learning and Doing Statistics," *International Statistical Review,* 1997.

[8] L. Bukovský, Množiny a všeličo okolo nich, UPJŠ v Košiciach, 1985.

[9] ORACLE, „Java Platform, Standard Edition (Java SE) Documentation," Oracle Corporation, 2024. [Online]. Available: https://docs.oracle.com/en/java/javase/. [Cit. 2026].

[10] OPENJFX, „JavaFX Documentation," Gluon & OpenJFX Project, 2024. [Online]. Available: https://openjfx.io/openjfx-docs/.

Zoznam použitých promptov

- **Štylistická úprava a odborná terminológia:** formulovanie matematických paradoxov a interpretácia výsledkov simulácií, Gemini 1.5 Flash, apríl 2026. Príklad zadania: „Uprav nasledujúci text do formálneho akademického štýlu bez zmeny významu.“
- **Analýza softvérovej architektúry:** konzultácia k popisu UML diagramov a štruktúry balíkov, Gemini 1.5 Pro, apríl 2026. Príklad zadania: „Navrhni osnovu kapitoly o softvérovej architektúre pre bakalársku prácu.“
- Formálna úprava bibliografie: generovanie citácií v štýle IEEE, Gemini 1.5 Flash, apríl 2026. Príklad zadania: „Vytvor citáciu v štýle IEEE pre tento zdroj.“
- **Preklad abstraktu:** odborný preklad do anglického jazyka so zachovaním terminológie, Gemini 1.5 Pro, apríl 2026. Príklad zadania: „Prelož tento abstrakt do angličtiny so zachovaním odbornej terminológie.“
- **Konzultácie k implementácii:** overenie postupov pri viacvláknovom spracovaní v JavaFX, Gemini 1.5, apríl 2026. Príklad zadania: „Vysvetli princíp viacvláknového spracovania v JavaFX na úrovni študenta.“
- **Optimalizácia vizualizácie dát:** návrhy na efektívne zobrazenie štatistických údajov, Gemini 1.5, apríl 2026. Príklad zadania: „Navrhni, ako vizualizovať veľké množstvo dát v grafe pre lepšiu čitateľnosť.“
- **Jazyková korektúra:** gramatická a štylistická úprava technického textu, Gemini 1.5, apríl 2026. Príklad zadania: „Skontroluj gramatiku a terminológiu v tomto odseku z oblasti informatiky.“

Zoznam príloh

Príloha A Projektová dokumentácia a zdrojové kódy

Príloha B Obsah DVD

Prílohy

Príloha A: Projektová dokumentácia a zdrojové kódy

- Elektronická verzia bakalárskej práce (PDF).
- Zdrojové kódy simulačnej aplikácie (Java, JavaFx).
- Spustiteľný súbor aplikácie (.jar).
- Online repozitár: https://github.com/Konecny18/bakalarkaVazni.git

Príloha B: Obsah pamäťového zariadenia

- Práca v elektronickej podobe
- Zdrojové kódy skriptov
- Dáta zo simulácie
- Šablóny Excel súborov